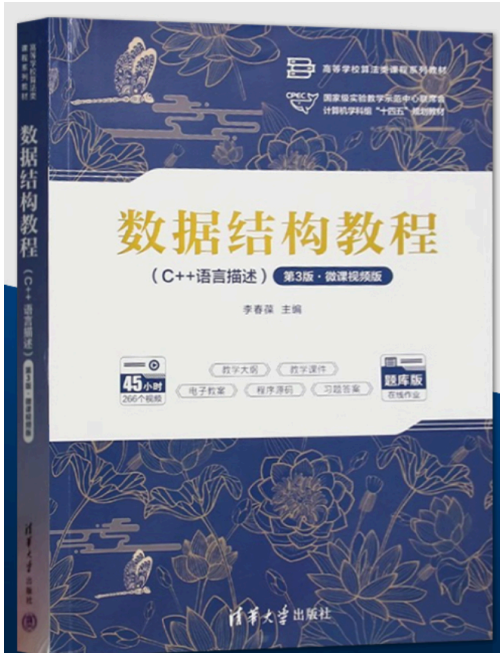
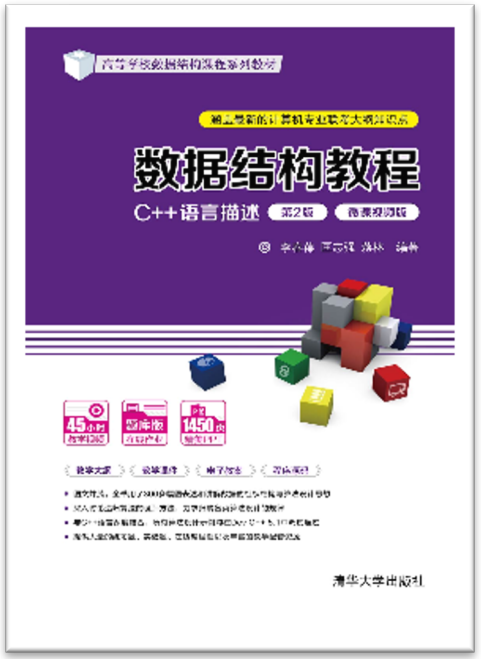
新版



ISBN：9787302685340

旧版



ISBN：9787302585329

《数据结构教程》(C++语言描述)第3版,李春葆等,清华大学出版社 2025

《数据结构教程（C++语言描述）学习与上机实验指导》

《数据结构教程（C++语言描述）在线编程题解析》

修改群名片：实名

希冀实验平台

https://cslabcg.whu.edu.cn/

- 账号已导入，首次登录密码位1234。
- 登陆后务必补充email，方便找回密码。
- 如果以前已经注册账号，密码不变。

本学期考核方式

1.课堂考核：20分

(1) 10分 期中考试。

(2) 10分 模拟机考自测训练（闭卷）

① **A班**：训练成绩在60分以上计一次有效成绩2.5分，有效计数达到4次，可得10分满分。从第4周开始。

② B班：训练成绩可加权，60分以上计一次满分成绩2.5分，60分以下按比例赋分；训练总成绩计算得分最高的4次，满分10分。从第4周开始。

③ 自测时间：6-17周（每周一次，时间待定）。

2.课后作业：15分

3.实验报告：15分

4.期末考试（闭卷）：50分

期末机考说明

期末考试（闭卷）：

A、B班均采用机考，满分为100分。

A班：

1. A班选择题共25题，1题2分，共50分，

2. 算法题3题，第1、2、3题分值分别为20、15、15。3道题难度依次为简单、中等、困难。算法题需自己编写完整的程序或函数，根据题目情况可设置步骤分，在指定步骤输出中间结果正确得步骤分。

B班：

1. B班选择题共30题，1题2分，共60分；。

2. 算法题3题，第1、2、3题分值分别为15、15、10。3道题难度依次为简单、简单、中等。第1题，题目给出大部分代码并描述解题思路，补全代码。第2、3题，和A班类似，需自己编写完整的程序或函数，根据题目情况可设置步骤分，在指定步骤输出中间结果正确得步骤分。

学习经验分享（1）

黄文婷

作为一个大学前没有编程基础，数学思维偏弱，但最终成绩还不错而且AB课都修了的过来人，谈数据结构的学习经验我好像确实有点话语权哈哈。

先说点经历向的。其实刚接触数据结构的时候它给我的感觉很难，理论知识很抽象，实操debug时也很痛苦费时间。课后作业经常一道题要几个小时，上课学的东西也不能很快理解。再加上期中考考的糟糕，当时差不多有了种零基础没有办法学好这门课的信念，就选择开启二周目了。在二周目的时候拜了了一个大学前也是零基础的大佬为师，经过了一个月的勤学好问后发现数据结构没那么难了，那种信念也消失了。

再说点建议向的。

1. 上课有些跟不上的话可以提前看PPT，PPT看不进的话可以试试看《大话数据结构》那本书。系统地听网课个人感觉很费时间也很难坚持，认真听课、遇到不理解的及时在CSDN上搜索理解就够了。

学习经验分享（1）

黄文婷

2. 刷题如果不知道从何下手的话可以直接刷**洛谷的官方题单**。刷题锻炼的主要是**构建算法、推测用例、调试错误的能力**，刷的题对应的知识点是否和课本的完全对应其实没太大影响，而且洛谷题单上普及难度的题和考试编程题的难度和风格是很相近的，我也是试了好几个不同的网站后自认为洛谷官方题单最适合新手宝宝体质。

3. 要熟练掌握书上有的经典算法（Kmp，Floyd…），有时考题会有明确的算法要求。这个其实考前复习的时候留意一下就差不多了，但不留意也许就寄了。

最后引用大佬的一句话“做任何事都会经历一个从蠢到聪明的过程”。相信自己、勤学多问肯定能做好哒。有疑问欢迎随时找我，我很乐意和大家交流探讨、共同探索解决问题的方案，千万不要嫌弃我不是大佬或者觉得不好意思呀。祝大家学习顺利～

学习经验分享（2）

金润禾

1 算法是基础

我认为，数据结构考察的就是两件事：

① 这个算法、数据结构的原理你懂不懂?
② 如果懂了，你能不能用代码实现出来?

对于第一件事，我们这门课程考察的算法和数据结构并没有特别高深，即便是最难以理解的算法 (比如 kmp) 一共也就那么几个，网上都有大量地剖析教程，只要多听课、多查资料、多思考都很容易能够理解。困扰大多数人的是第二件事情。

2 代码实现是核心

2.1 功利的路线规划

我认为，抛开这门课程不提，“大一下”这个时间段真的是提升代码水平的黄金时间。因为大家暂时没有科研的压力，有相对充裕的时间，同时这个学期有大量各种各样的小型算法竞赛 (蓝桥杯、天梯赛、睿抗编程赛...)，如果在这个时间段掌握了很好的代码实现能力，可以很容易在这些比赛中捞到一些奖项，对于大一学年的综合测评有很大的好处。

而这个学期结束后的暑假，将会迎来武大 ACM 集训队的暑期集训，在这时如果有能拿得出手的算法水平，就有机会去找到高年级的大佬们组队，在大佬的带领下拿到暑期集训的好名次，进而拿到下个学期 XCPC 区域赛的参赛资格。而在区域赛中，即便是拿到一块铜牌，对于综测、保研也是有很大帮助的事情。

2.2　对于数据结构课程

从实际的例子来看，我们的另一位黄助教用了一个学期的时间，在洛谷上做了一百多道题，就从算法几乎零基础走到了数据结构 A 满分的水平。所以我认为只要坚持不断提升自己的算法能力、代码实现能力，搞定数据结构这门课是捎带脚的事情。

2.3　具体做法

2.3.1　清晰地认识算法学习中遇到的困难

算法与其他领域相比，特点是入门很难，每一种算法在第一次应用的时候基本都会遇到各种各样的问题，很多人会因此感到畏惧。所以，我认为第一个要克服的就是**畏难情绪**。我们要习惯于几个小时甚至几天来思考同一个问题，广泛地阅读，看看前人是如何理解并应用这种算法的。通过这种方式学会的东西，可以说是一劳永逸的，以后再想用到它时就会信手拈来。最忌讳的事情就是**不会就抄**，如果是对照着别人写的正确代码进行编程，几乎学不到任何东西。

数据结构课上时间有限，讲的内容不够详细，所以如果对某些算法感到费解是很正常的，网上有很多优质的资源，比如 https://oi-wiki.org/ 这里可以查阅到我们会学到的所有算法，而且都附有很详细的讲解和例题，可以参考。

2.3.2　多做题

对于算法的学习来说，做题是必不可少的，所有的思维、实现的细节都只有在实战中才能得到验证。只完成数据结构作业中的那几道题目是远远不够的，这里推荐一些我一直在使用的做题平台：

- 洛谷：https://www.luogu.com.cn/ 里面有大量的专题训练以及模板题。
- cf：https://codeforces.com/ 在这个平台上会经常举办一些小比赛，通过比赛既能体会到限时编程的压力也能享受到 Accept 的快乐。不要被英文吓到，有翻译的办法：https://blog.csdn.net/2303_80346267/article/details/139204760。
- 牛客：https://ac.nowcoder.com/acm/contest/vip-index 这是国产的 cf，也会经常举办一些比赛，题面中文很友好，而且社区也是中文的，可以找到各种各样的题解。

对于算法题来说，有很多种做题方式：

- 自己思考 -> 自己实现 -> ac
- 自己思考 -> 放弃，看题解文字 -> 自己实现 -> ac

- 自己思考 -> 放弃，看题解文字 -> 放弃，看题解代码 -> 自己实现 -> ac
- 自己思考 -> 放弃，看题解文字 -> 放弃，看题解代码 -> 放弃，抄代码 -> ac

我认为，除了最后一种方式以外，前面三种都是可以接受的。虽然前面说了要习惯长时间思考同一个问题，但切记不要跟一道题目死磕。一旦发现自己实在没有思路，看题解或者询问别人是很合理的解决方案，但是一定一定要记得自己独立地把代码写出来。

2.3.3 养成好的编程习惯

一个良好的编程习惯是非常非常重要的，这是我的个人观点。所谓编程习惯包括“变量的命名方式”、“空格的使用方法”等等，这一点如果能在刚开始写代码的时候养成，会给以后省去非常非常多的麻烦。比如我的码风：

```
#include <bits/stdc++.h>
using namespace std;
using i64 = long long;

void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    multiset<int> s;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        s.insert(a[i]);
```

学习经验分享（2）

金润禾

3　结语

我认为，仅仅盯着数据结构的分数进行学习是动力不足的，只有出于对算法、编程的兴趣进行学习，才是长久且正确的。我相信，只要你能坚持学下去，通过数据结构的考试不会有任何问题。

以及，有任何问题都欢迎来向我提问，一定知无不答。

重要事项

- 课前预习，课后复习；
- 课下一定要重视编程练习，课后作业**独立完成**并计时，注重编程实操能力的锻炼和培养。
- 可以通过在线编程网站的补充练习逐步提升解题速度，通过机考自测查漏补缺；
- 课程学习的相关问题和讨论**请发在课程Q群**，请假事项请联系助教。课程组会定期统一组织线上或线下答疑；
- 善用**AI**工具，但要注意目标是自己能力的提升；

数据结构课程概述

信息的表达：如何高效存储及获取

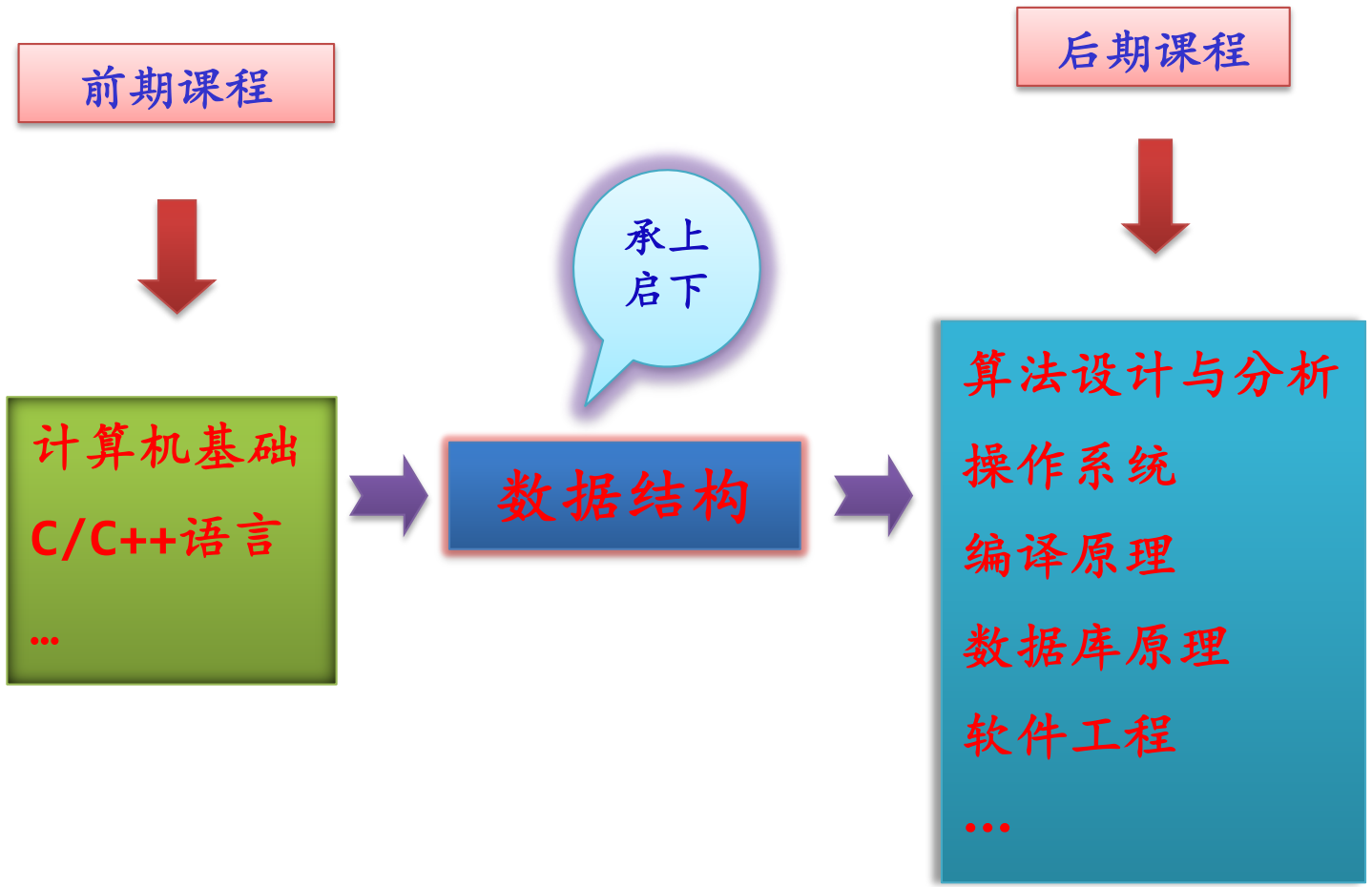
大数据：如何组织？如何处理？

1 “数据结构”课程的内容

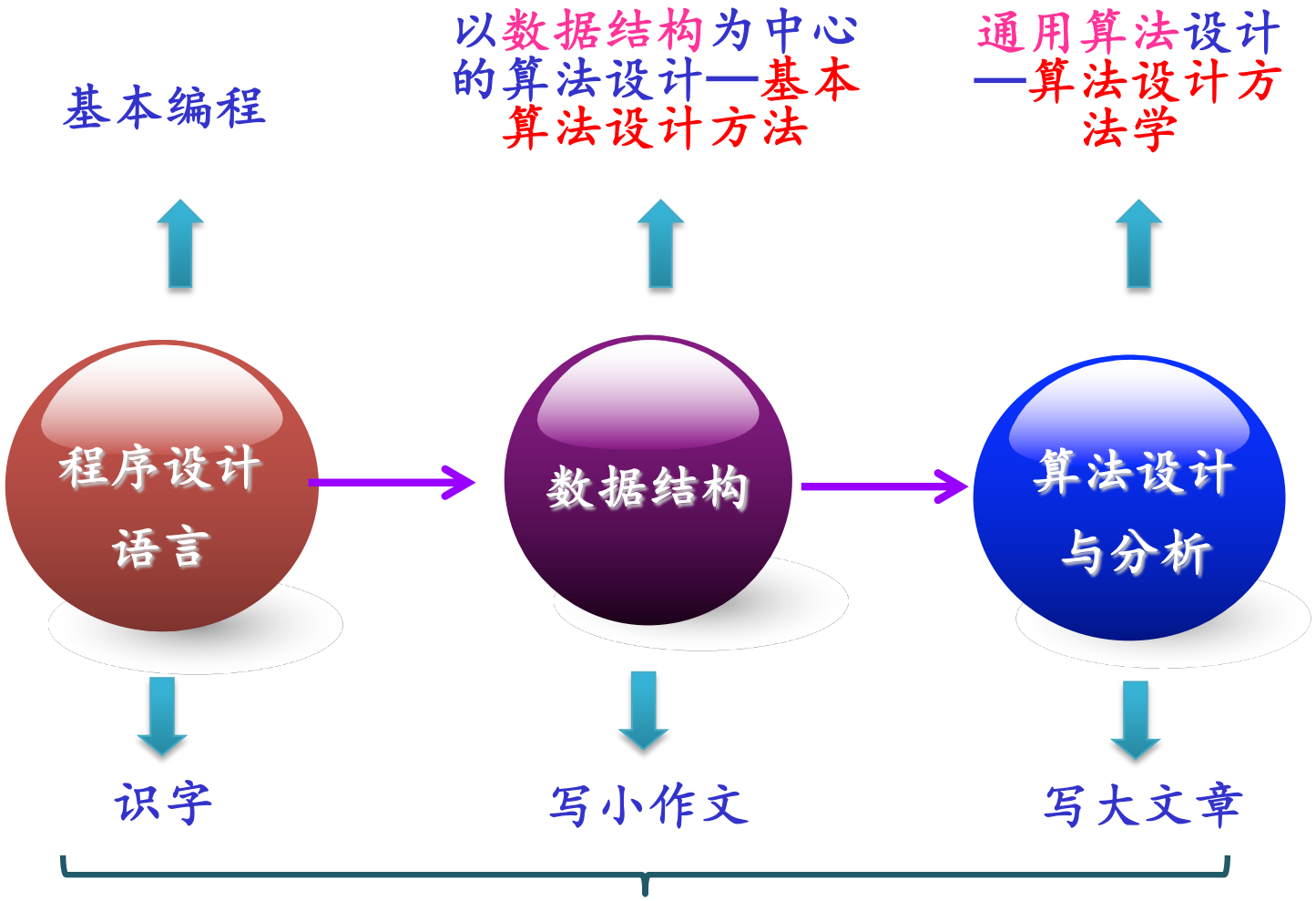
- 各种数据的逻辑结构描述
- 各种数据的存储结构表示
- 各种数据结构的运算定义
- 设计实现运算的算法
- 分析算法的效率

基本数据组织和
数据处理方法

2 “数据结构”在计算机课程体系中的地位



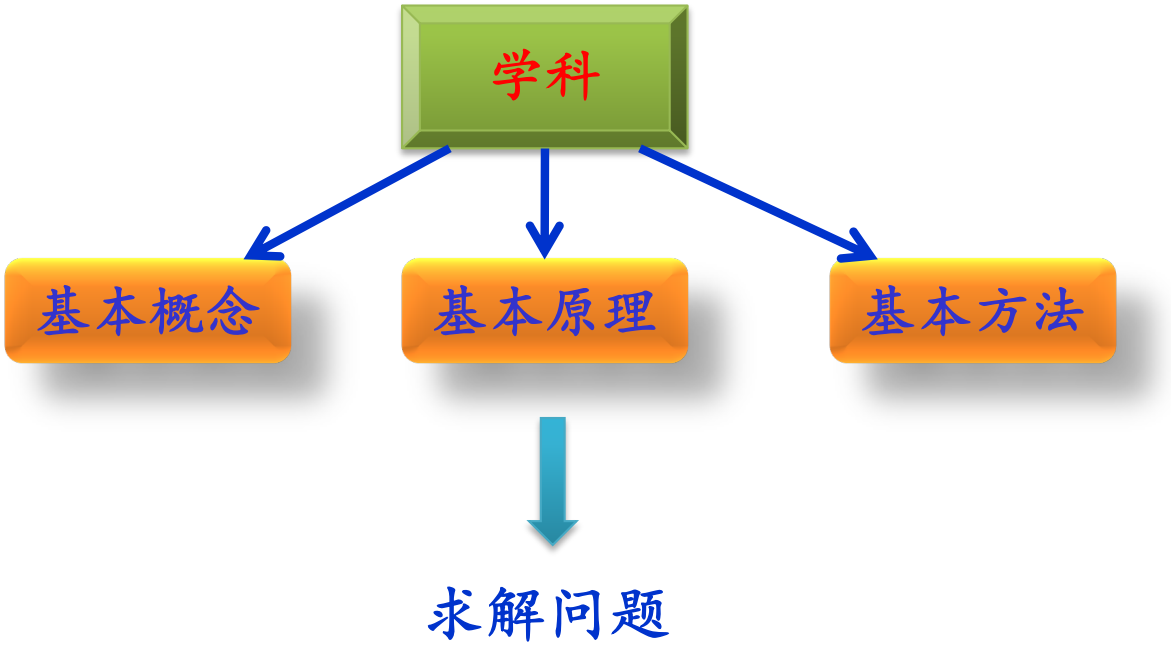
3 “数据结构”与程序设计类课程的关系



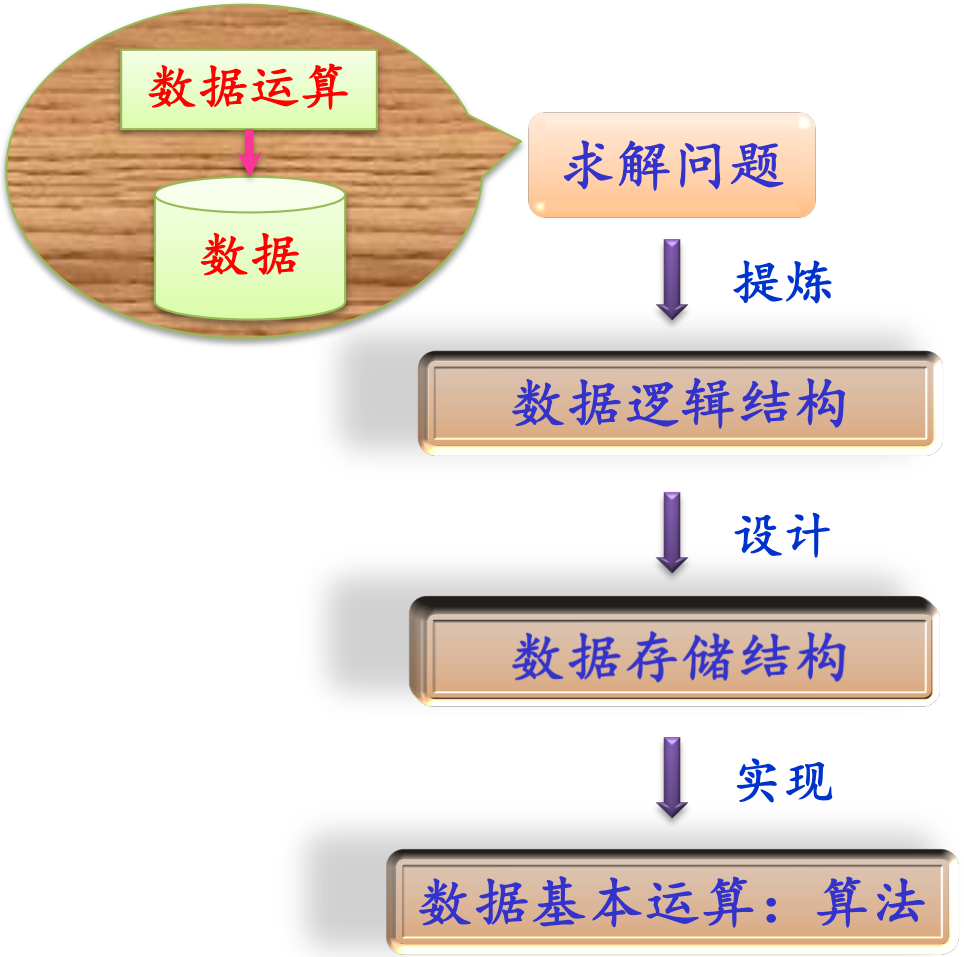
与语文学习过程类比

4 “数据结构”的学习目标

❶ 掌握数据结构的基本概念、基本原理和基本方法。



❷ 掌握数据的逻辑结构、存储结构及基本运算的实现过程。



❸ 掌握算法基本的时间复杂度与空间复杂度的分析方法，能够设计出求解问题的高效算法。

同一求解问题通常有多种实现算法，通过时间复杂度与空间复杂度的分析，找出最好的实现算法。

例如，求1 + 2 + … n。

算法1:

```
int fun1(int n)
{  int i, s=0;
   for (i=1;i<=n;i++)
      s+=i;
   return s;
}
```

算法2:

```
int fun2(int n)
{
   return (n+1)*n/2;
}
```

算法分析

算法2好于算法1

5 “数据结构”的学习方法

❶ 理解各种数据结构的逻辑特性和存储结构设计

逻辑特性

映射：计算机中的表示

存储结构

❷ 掌握各种数据结构算法设计的基本方法

逻辑特性

↓ 映射：计算机中的表示

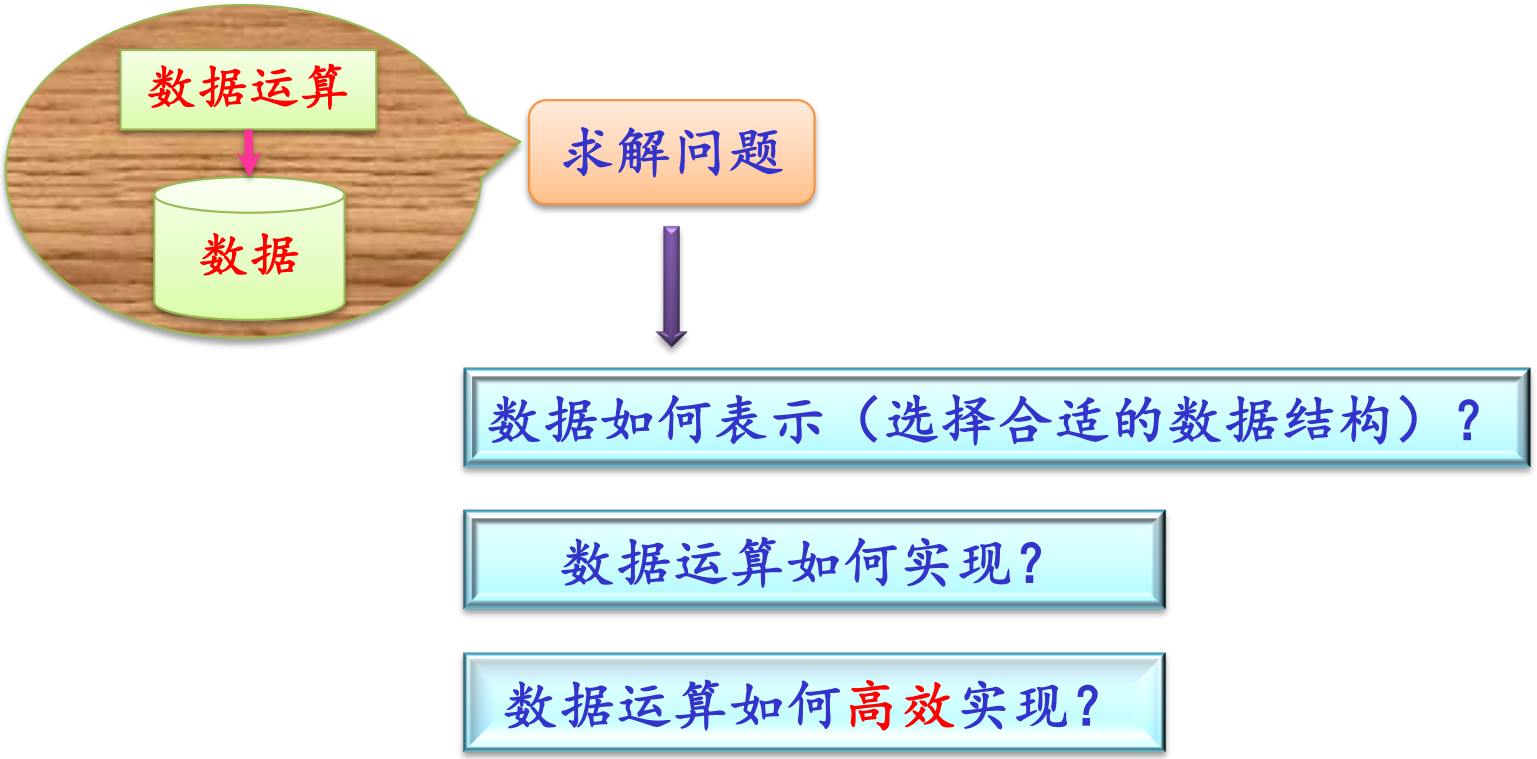
存储结构

↓ 运算实现

算法设计

只有掌握了数据的存储结构表示，才能在此之上设计算法。

❸ 利用各种数据结构来求解实际问题



❹ 演绎和归纳相结合

培根：方法是旧的，问题是新的

推荐练习网站 https://www.luogu.com.cn/

推荐练习网站 https://www.leetcode.cn

推荐练习网站

http://poj.org/

推荐练习网站

http://acm.hdu.edu.cn/

第1章 绪论

提纲

CONTENTS

1.1 什么是数据结构

1.2 算法及其描述

1.3 算法分析

1.4 数据结构的目标

1.1 什么是数据结构

1.1.1 数据结构的定义

用计算机解决一个具体问题需要从数据入手，数据的几个概念：

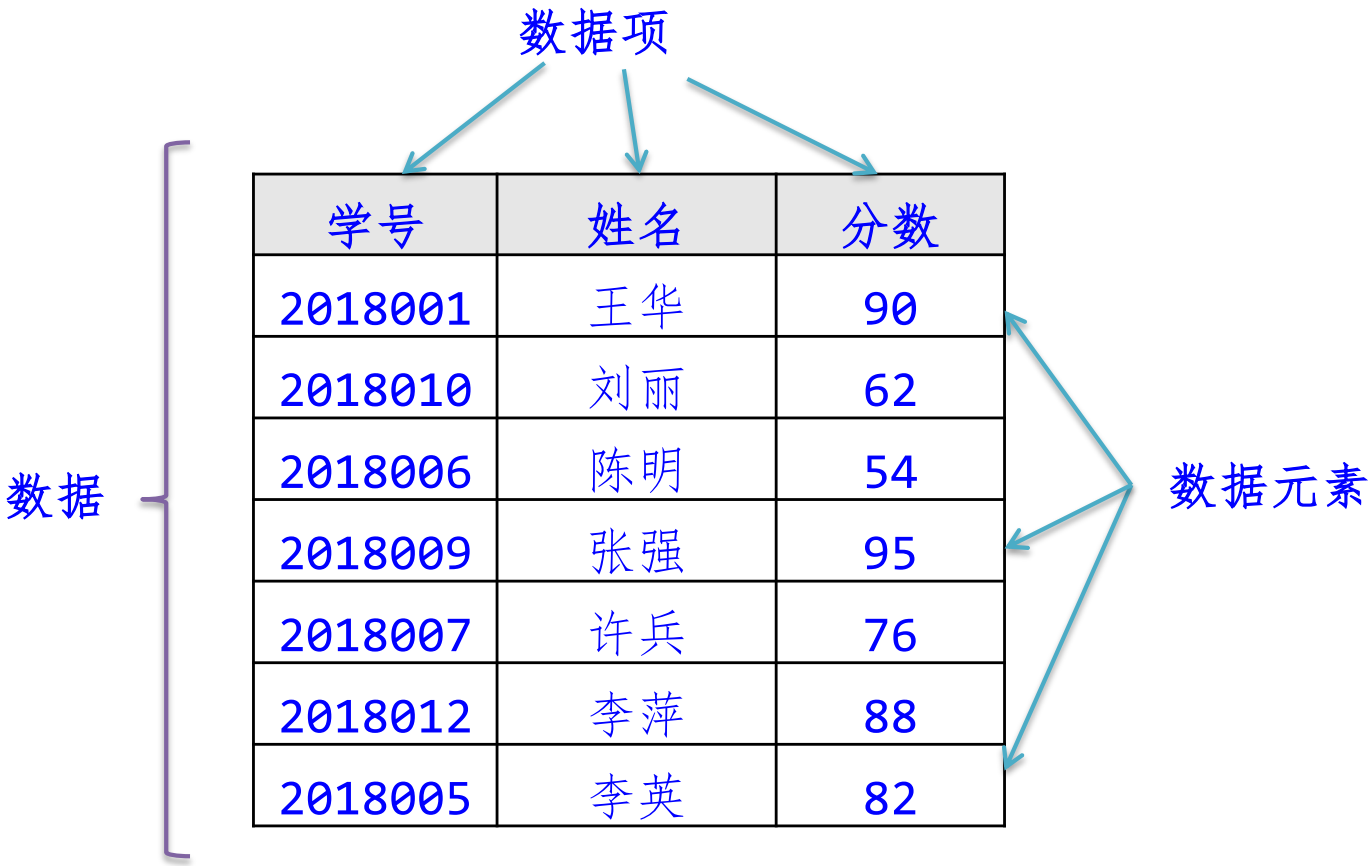
数据是描述客观事物的数、字符以及所有能输入到计算机中并被计算机程序处理的符号的集合。

数据元素是数据的基本单位（例如，A班中的每个学生记录都是一个数据元素），也就是说数据元素是组成数据的、有一定意义的基本单位，在计算机中通常作为整体处理

数据项是具有独立含义的数据最小单位，也称为成员或域（例如，A班中每个数据元素即学生记录是由学号、姓名、性别和班号等数据项组成）。

结构化数据

高等数学成绩表



学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

数据对象是性质相同的有限个数据元素的集合，它是数据的一个子集。

如大写字母数据对象是集合$C=\{'A', 'B', 'C', \cdots, 'Z'\}$；

1～100的整数数据对象是集合$N=\{1, 2, \cdots, 100\}$。

默认情况下，数据结构中的数据都指的是数据对象。

数据结构是指所涉及的数据元素以及数据元素之间的关系，可以看作是相互之间存在着特定关系的数据元素的集合。可时把数据结构看成是带结构的数据元素的集合。

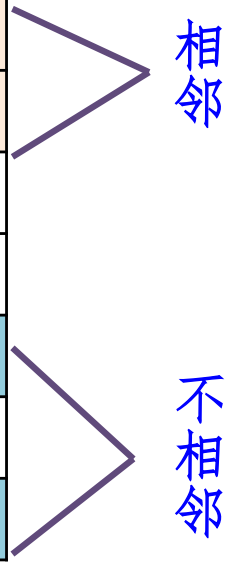
数据结构 = **数据对象** + **结构**

相同性质的数据元素的集合

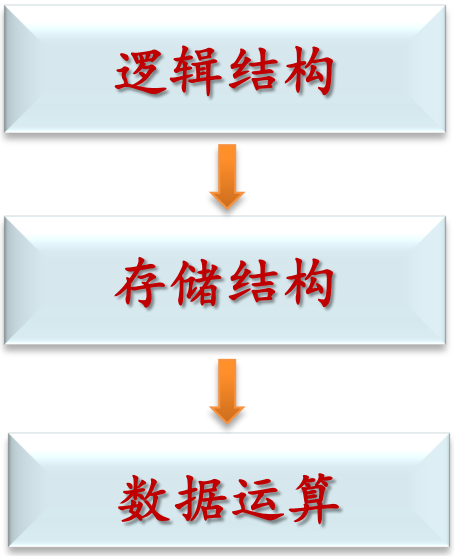
数据元素之间的关系构成结构

数据结构中讨论的元素关系主要是指**相邻关系**或**邻接关系**。

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82



一个数据结构的几个方面：



- 数据元素之间的逻辑关系 ⇨ 数据的逻辑结构。
- 数据元素及其关系在计算机存储器中的存储方式 ⇨ 数据的存储结构（或物理结构）。
- 施加在该数据上的操作 ⇨ 数据运算。

1.1.2 数据的逻辑结构

- 数据的逻辑结构是面向用户的，它反映数据元素之间的逻辑关系而不是物理关系。
- 数据的逻辑结构是独立于计算机的。

逻辑结构

1．逻辑结构的表示

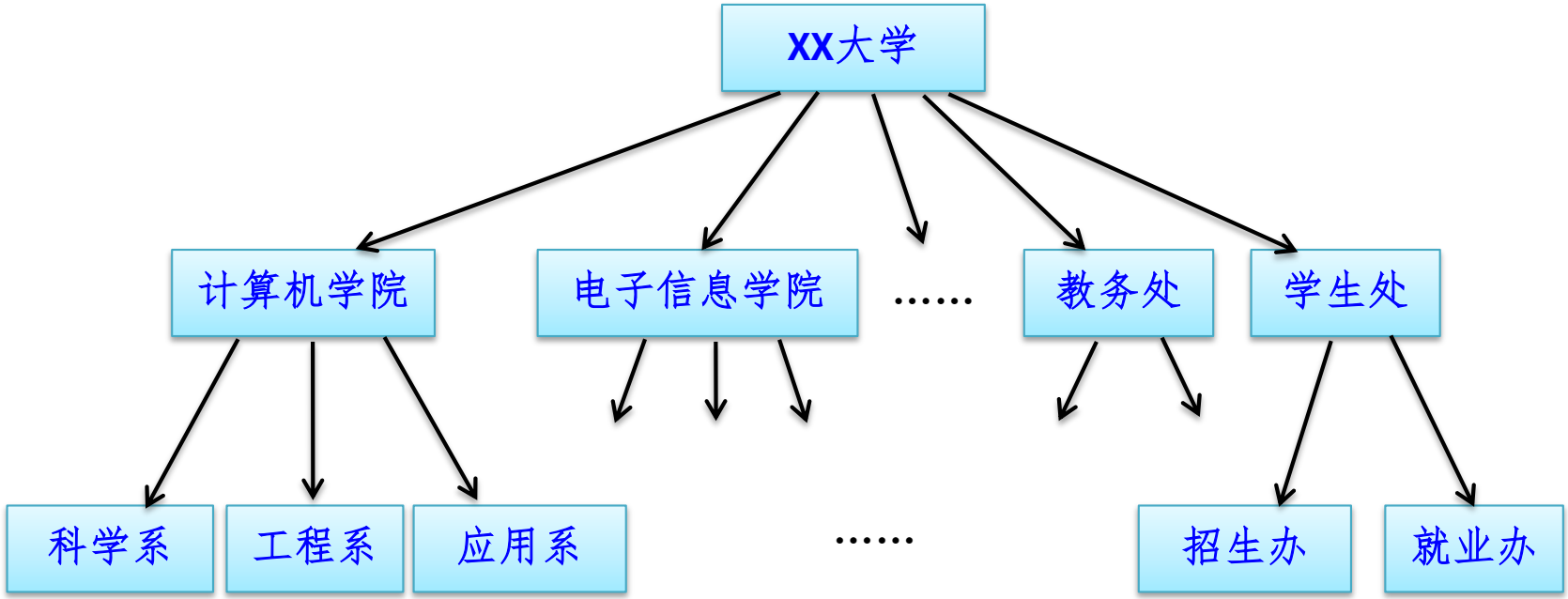
由于数据逻辑结构是面向用户的，可以采用表格、图等用户容易理解的形式表示。

例1.1

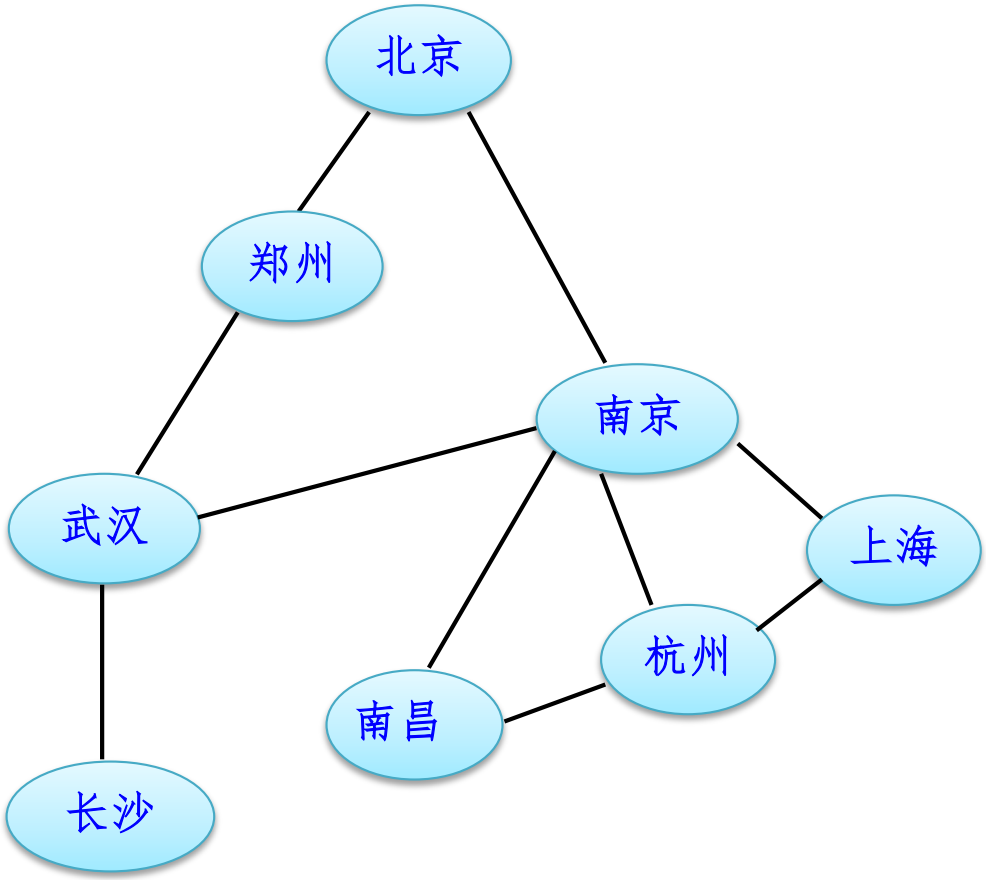
高等数学成绩表

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

例1.2



例1.3



为了更通用地描述数据的逻辑结构，通常采用二元组表示数据的逻辑结构，一个**二元组**如下：

$B=(D, R)$

其中，B是一种逻辑数据结构，D是数据元素的集合，在D上数据元素之间可能存在多种关系，R是所有关系的集合。即：

$D=\{d_i \mid 0 \leqslant i \leqslant n-1, n \geqslant 0\}$

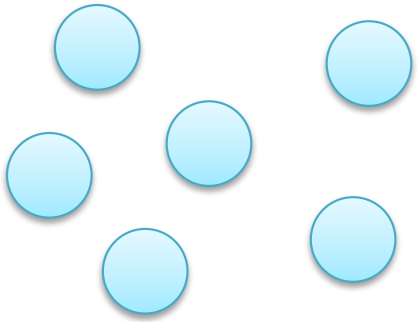
$R=\{r_j \mid 1 \leqslant j \leqslant m, m \geqslant 0\}$

$R=\{r_j \mid 1 \leqslant j \leqslant m,\ m \geqslant 0\}$

- R中的某个关系r_j（$1 \leqslant j \leqslant m$）是序偶的集合。
- 对于r_j中的任一序偶$\langle x, y\rangle$（$x, y \in D$），把x叫做序偶的第一元素，把y叫做序偶的第二元素，又称序偶的第一元素为第二元素的前驱元素，称第二元素为第一元素的后继元素。如在$\langle x, y\rangle$的序偶中，x为y的前驱元素，而y为x的后继元素。
- 若某个元素没有前驱元素，则称该元素为开始元素；若某个元素没有后继元素，则称该元素为终端元素。
- 对于对称序偶，即满足这样的条件：若$\langle x, y\rangle \in r$（$r \in R$），则$\langle y, x\rangle \in r$（$x, y \in D$），可用圆括号代替尖括号，即$(x, y) \in r$。

2．逻辑结构的类型

- **集合**：结构中数据元素之间除了“同属于一个集合”的关系外，没有其他关系，与数学中的集合概念相同。

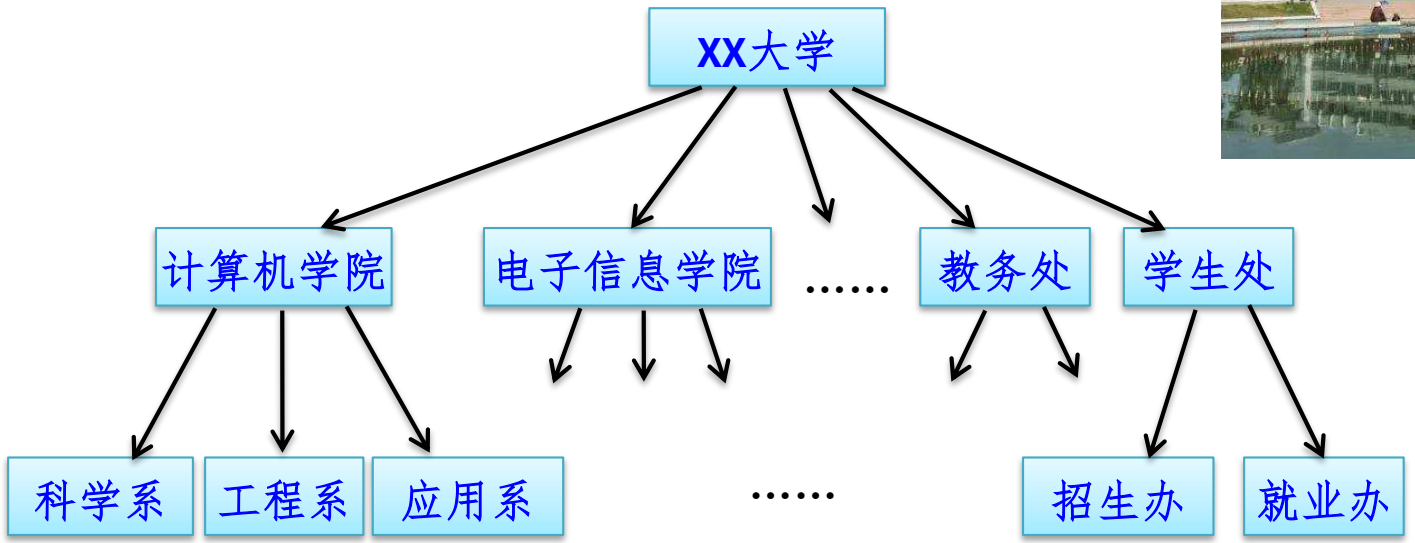


- **线性结构**：若结构是非空的，则有且仅有一个开始元素和终端元素，并且所有元素最多只有一个前驱元素和一个后继元素。

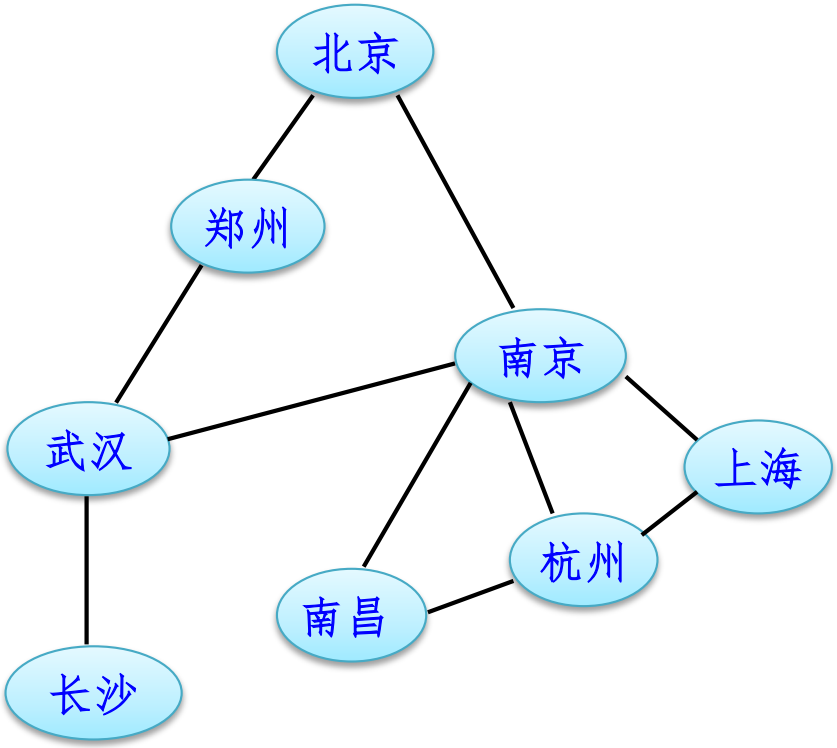
高等数学成绩表

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

- **树形结构**：若结构是非空的，则有且仅有一个元素为开始元素（也称为根结点），可以有多个终端元素，每个元素有零个或多个后继元素，除开始元素外每个元素有且仅有一个前驱元素。



- **图形结构**：若结构是非空的，则每个元素可以有多个前驱元素和多个后继元素。

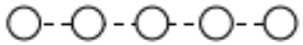


数据的逻辑结构

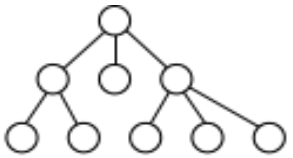
逻辑上，数据元素之间的关系只有4种：无关系、一对一关系、一对多关系、多对多关系，这4种逻辑关系总称为数据的逻辑结构。

- 集合：包含的所有数据元素之间无关系，即数据元素之间的次序是任意的。

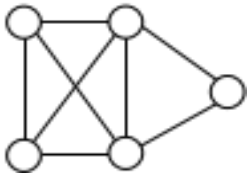
- 线性结构：包含的数据元素之间存在一对一的关系，即数据元素之间构成一个有序序列。



- 树形结构：包含的数据元素之间存在一对多的关系，即数据元素之间形成一个层次关系。

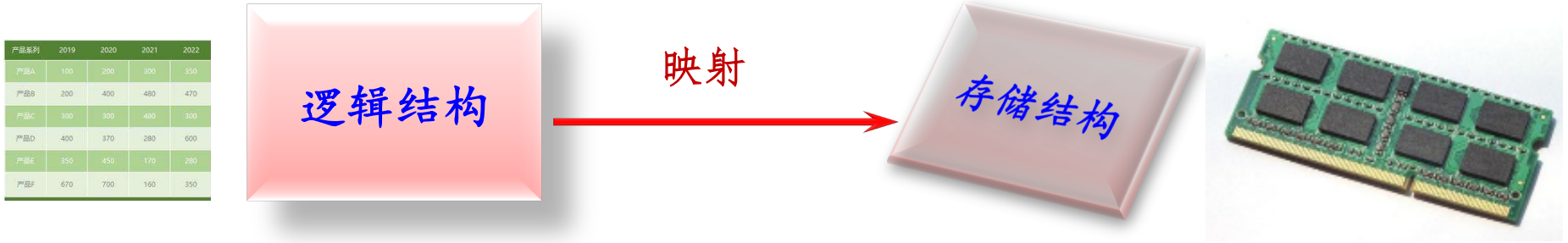


- 图形结构：包含的数据元素（结点）之间存在多对多的关系，即图中每个数据元素的前驱和后继数目都不限。



1.1.3 数据的存储结构

数据在计算机存储器中的存储方式就是存储结构。它是面向程序员的。



设计存储结构的这种映射应满足两个要求：

- 存储所有元素
- 存储数据元素间的关系

【例1.5】对于表1.1所示高等数学成绩表，设计多种存储结构，并讨论各种存储结构的特性。

高等数学成绩表

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

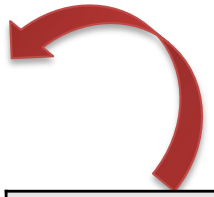
存储结构1：用C++语言中的结构体数组来存储高等数学成绩表，设计其元素类型Stud1如下：

```
strcut Stud1                                          //学生成绩元素类型
{    int no;                                          //存放学号
     string name;                                     //存放姓名
     int score;                                       //存放分数

     Stud1() {}                                       //构造函数
     Stud1(int no1,string name1,int score1)           //重载构造函数
     {    no=no1;
          name=name1;
          score=score1;
     }
};
```

用data数组（所有的元素类型均为Stud1）存放高等数学成绩表：

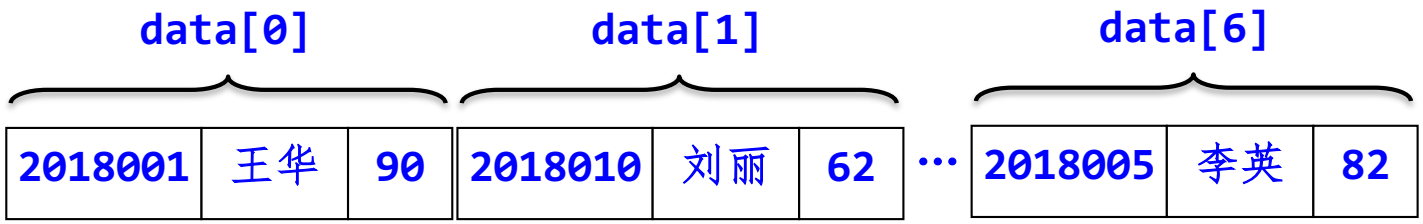
```
void Create()          //创建高数成绩顺序表
{  data[0]=Stud1(2018001,"王华",90);
   data[1]=Stud1(2018010,"刘丽",62);
   data[2]=Stud1(2018006,"陈明",54);
   data[3]=Stud1(2018009,"张强",95);
   data[4]=Stud1(2018007,"许兵",76);
   data[5]=Stud1(2018012,"李萍",88);
   data[6]=Stud1(2018005,"李英",82);
   length=7;
}
```



学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

映射



该存储结构的特性：

- 所有元素存放在一片地址连续的存储单元中。
- 逻辑上相邻的元素在物理位置上也是相邻的，所以不需要额外空间表示元素之间的逻辑关系。

称为**顺序存储结构**。

优点

- 存储效率高
- 实现元素的随机存取

缺点

- 不便于元素的插入和删除

存储结构2：用C++语言中的单链表来存储高等数学成绩表，设计其结点类型Stud2如下：

```
struct Stud2                                          //学生单链表结点类型
{  int no;                                            //存放学号
   string name;                                       //存放姓名
   int score;                                         //存放分数
   Stud2* next;                                       //存放下一个结点指针

   Stud2(int no1,string name1,int score1)             //重载构造函数
   {  no=no;
      name=name1;
      score=score1;
      next=NULL;
   }
};
```

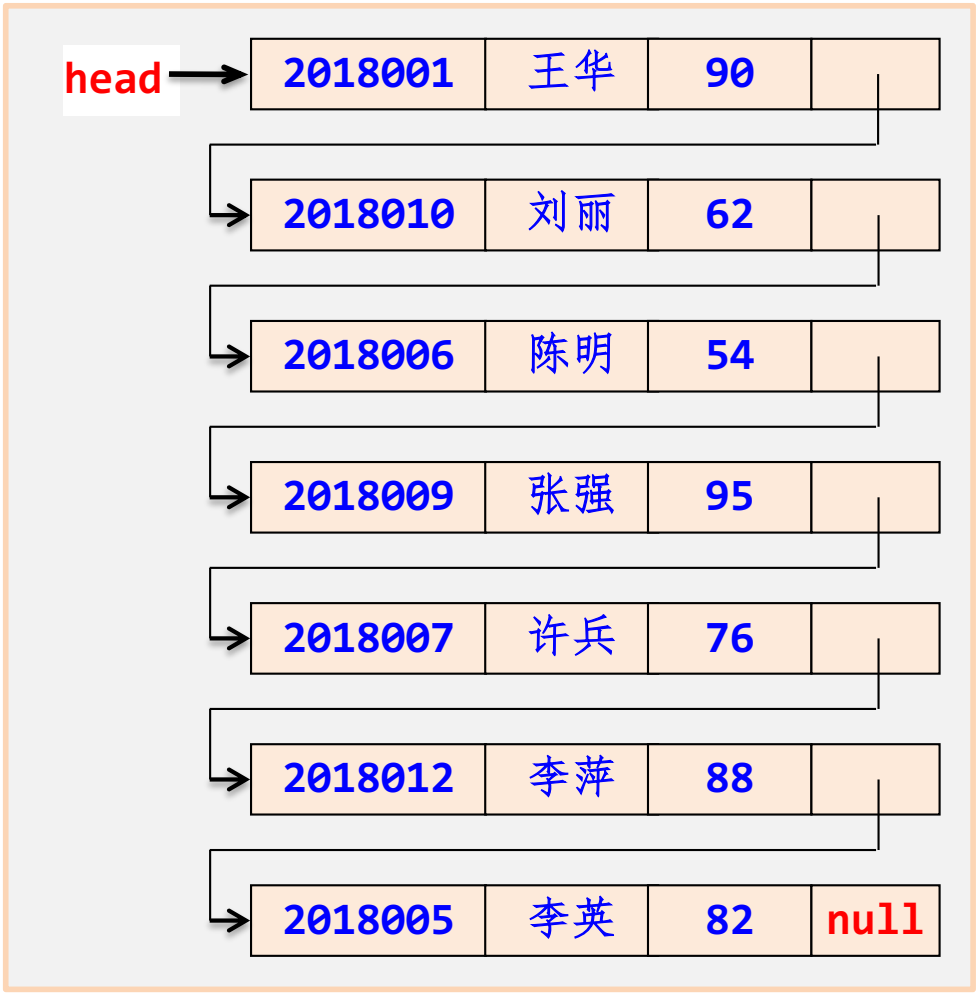
建立一个用于存放高等数学成绩表的单链表（开始结点为head）如下：

```
void Create()                                    //创建高数成绩单链表
{   Stud2* p2,*p3,*p4,*p5,*p6,*p7;
    head=new Stud2(2018001,"王华",90);//单链表首结点
    p2=new Stud2(2018010,"刘丽",62);  //建立其他结点
    p3=new Stud2(2018006,"陈明",54);
    p4=new Stud2(2018009,"张强",95);
    p5=new Stud2(2018007,"许兵",76);
    p6=new Stud2(2018012,"李萍",88);
    p7=new Stud2(2018005,"李英",82);
    head->next=p2;                               //建立结点之间的关系
    p2->next=p3;
    p3->next=p4;
    p4->next=p5;
    p5->next=p6;
    p6->next=p7;
    p7->next=NULL;                               //尾结点的next置为空
}
```

建立每个元素的结点

建立结点之间关系以表示对应元素的逻辑关系

用head唯一标识单链表



优点

- 便于元素的插入和删除

缺点

- 存储空间利用率低
- 不能随机存取

这种存储结构的特性：

- 数据元素存放在任意的存储单元中，这组存储单元可以是连续的，也可以是不连续的。
- 通过指针域来反映数据元素的逻辑关系。

称为**链式存储结构**。

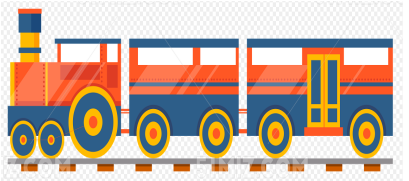
数据的存储结构

数据的存储结构（即数据的物理结构）：数据的逻辑结构在计算机内的存储方式。

- 顺序存储：将所有的数据元素存放在一段连续的存储空间中，数据元素的存储位置反应了它们之间的逻辑关系

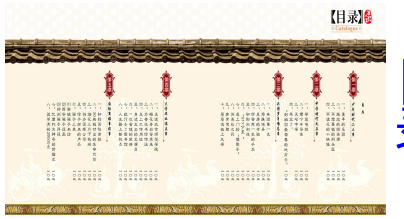
座位

- 链式存储：逻辑上相邻的数据元素不需要在物理位置上也相邻，也就是说数据元素的存储位置可以是任意的



列车

- 索引存储：在存储数据元素的同时还增加了一个索引表。索引表中的每一项包括关键字和地址，关键字是能够唯一标识一个数据元素的数据项，地址是指向数据元素的存储地址



目录

- 散列存储（即哈希存储）：将数据元素存储在一个连续区域，每一个数据元素的具体存储位置是根据其关键字的值，并通过散列（哈希）函数直接计算出来的

邮政编码

在软件开发中，人们设计了各种存储结构。归纳为4种基本的存储结构。

- 顺序存储结构
- 链式存储结构
- 索引存储结构
- 哈希（散列）存储结构

1.1.4 数据的运算

- 将数据存放在计算机中的目的是为了实现一种或多种运算。
- 运算包括功能描述（或运算功能）和功能实现（或运算实现）。
- 前者是基于逻辑结构的，是用户定义的，是抽象的。
- 后者是基于存储结构的，是程序员用计算机语言或伪码表示的，是详细的过程，其核心是设计实现某一运算功能的处理步骤，即算法设计。

例如，对于高等数学成绩表这种数据结构，可以进行一系列的运算：

- 增加一个学生成绩记录
- 删除一个学生成绩记录
- 求所有学生的平均分
- 查找序号为i的学生分数等。

学号	姓名	分数
2018001	王华	90
2018010	刘丽	62
2018006	陈明	54
2018009	张强	95
2018007	许兵	76
2018012	李萍	88
2018005	李英	82

同一运算，在不同存储结构中的实现过程是不同的。

例如，查找序号为i的学生分数，其本身就是运算的功能描述。但在顺序存储结构和链式存储结构中的实现过程不同的。

在顺序存储结构即data数组中实现查找

```
int Findi(int i)                          //查找序号为i的学生分数
{  if (i<0 || i>=length)                  //i错误时返回-1
       return -1;
   return data[i].score;                  //i正确时返回分数
}
```

在链式存储结构即head单链表中实现查找：

```
int Findi(int i)                          //查找序号为i的学生分数
{  if (i<0) return -1;
   int j=0;
   Stud2* p=head;                         //p指向第一个结点
   while (j<i && p!=NULL)
   {  j++;
      p=p->next;
   }

   if (p==NULL)                           //i错误时返回-1
      return -1;
   else                                   //i正确时返回其分数
      return p->score;
}
```

提示

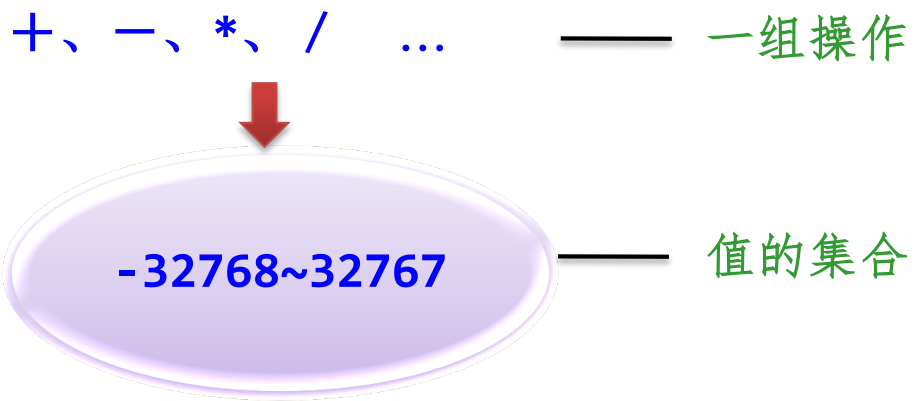
- 同一逻辑结构可以对应多种存储结构。
- 同样的运算，在不同的存储结构中，其实现过程是不同的。

1.1.5 数据结构和数据类型

1. 数据类型

数据类型是一组性质相同的值的集合和定义在此集合上的一组操作的总称。

例如，C++中的short int 就是整型数据类型。



- 数据结构是指计算机处理的数据元素的组织形式和相互关系，而数据类型是某种程序设计语言中已实现的数据结构。
- 在程序设计语言提供的数据类型支持下，就可以根据从问题中抽象出来的各种数据模型，逐步构造出描述这些数据模型的各种新的数据结构。

2．抽象数据类型

抽象数据类型（ADT）指的是从求解问题的数学模型中抽象出来的数据逻辑结构和运算（抽象运算），而不考虑计算机的具体实现。

ADT基本格式

```
ADT 抽象数据类型名
{   数据对象：数据对象的声明
    数据关系：数据关系的声明
    基本运算：基本运算的声明
} ADT 抽象数据类型名
```

【例1.6】构造集合ADT Set，假设其中元素为整型，遵循标准数学定义，基本运算包括：

求集合长度、求第i个元素、判断一个元素是否属于集合、向集合中添加一个元素、从集合中删除一个元素、复制集合和输出集合中所有元素。

另外增加3个集合运算：

求两个集合并Union、集合交Inter和集合差Diff。

```
ADT Set                                          //集合的抽象数据类型
{   数据对象:
      data={d_i | 0≤i≤size-1 }                   //存放集合中元素
    数据关系:
      无
    基本运算:
       getsize()                                 //返回集合的长度
       get(int i)                                //返回集合的第i个元素
       IsIn(E e)                                 //判断e是否在集合中
       add(E e)                                  //将元素e添加到集合中
       delete(E e)                               //从集合中删除元素e
       Copy(s)                                   //返回当前集合的复制集合
       display()                                 //输出集合中的元素
       Union(Set s2)                             //求s3=s1Us2 (s1为当前集合)
       Inter(Set s2)                             //求s3=s1∩s2 (s1为当前集合)
       Diff(Set s2)                              //求s3=s1-s2 (s1为当前集合)
} ADT Set
```

运算功能描述

Complex　　ADT

编程实现该数据结构

抽象数据类型实质上就是对一个求解问题的形式化描述（与计算机无关），程序员可以在理解基础上实现它。

1.2 算法及其描述

1.2.1 什么是算法

算法是对特定问题求解步骤的一种描述，它是指令的有限序列。

算法具有以下五个重要的特性

- **有穷性**。指算法在执行有限的步骤之后，自动结束而不会出现无限循环，并且每一个步骤在可接受的时间内完成。
- **确定性**。对于每种情况下执行的操作，在算法中都有确定的含义，不会出现二义性。并且在任何条件下，算法都只有一条执行路径。

- **可行性**。算法的每条指令都是可执行的，即便人借助纸和笔都可以完成。

求最大公约数算法！

设两数为a、b(a≥b)，求a和b最大公约数（a，b）的步骤如下：

（1）用a除以b(a≥b)，得a÷b=q..r1（r1≥0）。

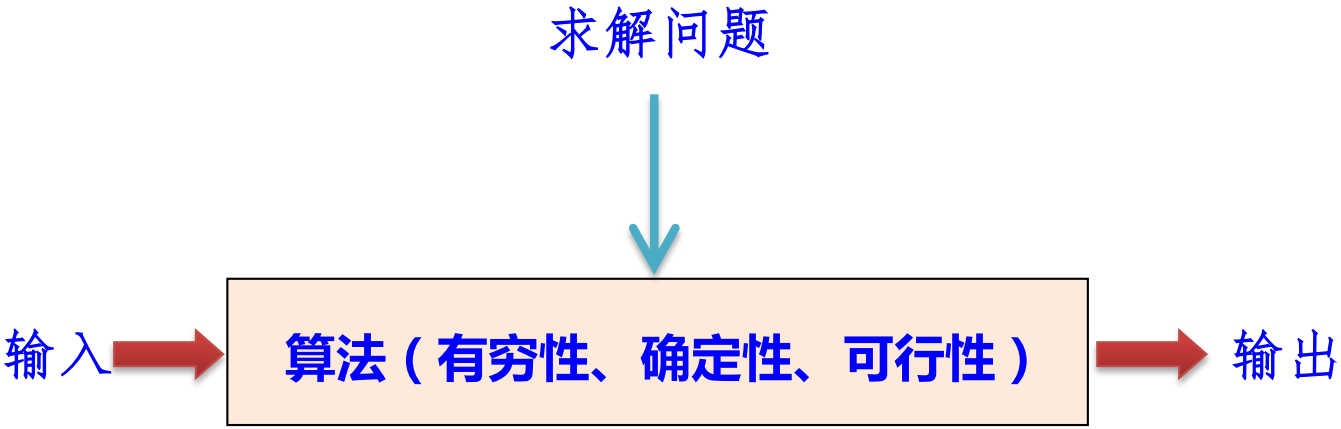
（2）若r1=0，则（a，b）=b，结束。

（3）若r1≠0，则再用b除以r1，得b÷r1=q..r2（r2≥0）。

（4）若r2=0，则（a，b）=r1，结束；若r2≠0，则继续，… ，如此下去，直到能整除为止。

其最后一个余数为0的除数即为（a，b）的最大公约数。

- **输入性**。算法有零个或多个输入。大多数算法中输入参数是必要的，但对于较简单的算法，如计算1+2的值，不需要任何输入参数，因此算法的输入可以是零个。
- **输出性**。算法至少有一个或多个输出。算法用于某种数据处理，如果没有输出，这样的算法是没有意义的，算法的输出是和输入有着某些特定关系的量。


求解问题
输入
算法（有穷性、确定性、可行性）
输出

【例1.7】考虑下列两段描述：

（1）描述一

```
void exam1():
{   int n=2;
    while(n%2==0)
        n=n+2;
    cout << n << endl;
}
```

（2）描述二

```
void exam2():
{   int x=0,y=0;
    x=5/y
    cout << x << endl;
}
```

这两段描述均不能满足算法的特征，试问它们违反了哪些特性？

有穷性

可行性

1.2.2 算法描述

通常算法用一个或者几个函数（或者方法）描述，其一般格式：

```
bool 算法对应的函数或者方法名(形参列表):
{    //临时局部变量的定义
     //实现由输入参数到输出参数的操作
       …
}
```

函数体

- 函数的返回值通常为布尔类型，表示算法是否成功执行。
- 形参列表表示算法的参数，由输入参数和输出参数构成。
- 函数体实现算法的功能。

【例1.8】求和问题是当$n \geqslant 1$时求$s=1+2+\cdots+n$。设计对应的算法。

- 输入参数为n，操作结果为s。
- 初始条件是$n \geqslant 1$。
- 当初始条件不满足，返回False，否则计算出s并返回True。

求解算法1

```
bool Sum1(int n,int& s)                    //求和算法1
{   if (n<1) return false;
    s=n*(n+1)/2;
    return true;
}
```

```
int main()
{   int n=-5,s;
    if (Sum1(n,s))
        printf("1到%d的和=%d\n",n,s);
    else
        printf("参数n错误\n");
    return 0;
}
```

在有些情况下可以直接用算法的返回值来区分输入参数的正确性。

求解算法2

```
int Sum2(int n)                    //求和算法2
{  if (n<1) return -1;
   return n*(n+1)/2;
}
```

```
int main()
{  int n=5;
   int s=Sum2(n);
   if (s!=-1)
      printf("1到%d的和=%d\n",n,s);    //输出：1到5的和=15
   else
      printf("参数n错误\n");
   reurn 0;
}
```

在用C++语言描述算法时，用throw语句抛出异常。

求解算法3

```
int Sum3(int n)                          //求和算法3
{  if (n<1)
      throw "参数n错误";                  //抛出异常信息
   return n*(n+1)/2;
}
```

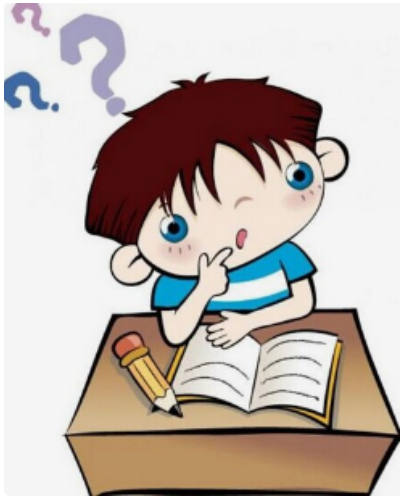
```
int main()
{   int n=-5;
    printf("1到%d的和=%d\n",n,Sum3(n));
    return 0;
}
```

1.2.3 C++语言描述算法要点

1．C++的数据类型

2．C++函数的参数传递

3．C++中的模板设计



自学

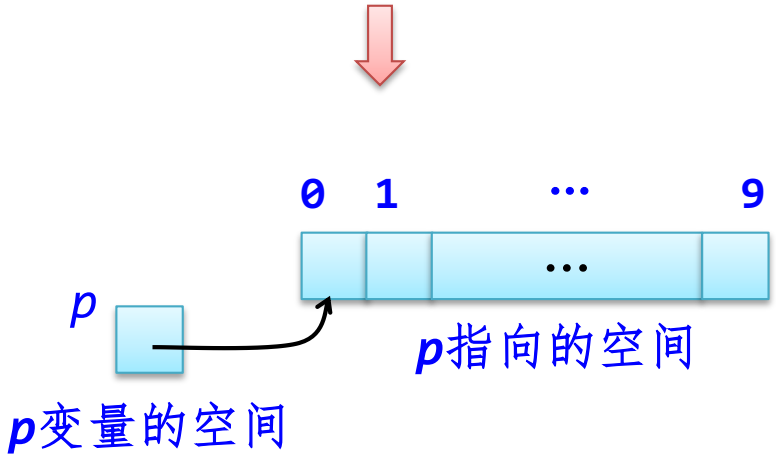
1.2.3 C++语言描述算法要点

1. C++的数据类型

- C++基本数据类型
  - int
  - float
  - double
  - char

C++指针类型

```
int* p=new int[10];
```



2．C++函数的参数传递

- 按值传递

```
void swap1(int x,int y)
{   int tmp=x;
    x=y;
    y=tmp;
}
```

单向值传递：a→x，b→y

```
int main()
{   int a=1，b=2;
    swap1(a,b);
    printf("a=%d,b=%d\n",a,b);
    return 0;
}
```


？ a=1,b=2

引用传递

数据类型&　引用名=关联变量名;

```
double d=10;                    //定义int型变量d
double& rd=d;                   //定义d的引用变量rd
```

（1）引用参数和实参共享相同的存储空间，所以对形参的修改会影响对应实参的值，可以理解为此时实参和形参是双向值传递。

```
void swap2(int& x,int& y)
{   int tmp=x;
    x=y;
    y=tmp;
}
```

引用传递： a↔x，b↔y

```
int main()
{   int a=1, b=2;
    swap2(a,b);
    printf("a=%d,b=%d\n",a,b);
    return 0;
}
```

a=2,b=1 ?

（2）由于引用参数本身并不创建新的存储空间，所以当调用函数时传递数据量较多的情况下可以尽可能采用引用参数。

```
int fun(MyClass& s)
{  int s;
    …
   return s;
}
```

引用参数s并不是为了实现双向传递而是节省栈空间

3．C++中的模板设计

函数模板

```
template 类型形参表
返回类型 函数名(形参表)
{
    函数体;
}
```

```
template <typename T>
T abs(T x)
{  if (x<0) return -x;
   return x;
}
```

```
int main()
{  int x=abs(-5);
   double d=abs(-1.25);
   return 0;
}
```

类模板

```
template 类型形参表
class 类模板名
{   类模板实现语句;
    …
};
```

类模板名<类型实参表> 对象表;

模板类

【例1.9】分析以下程序的功能。

```
template <typename T>
class Array                          //定义类模板Array<T>
{  T* data;                          //T为类型参数，data为指针变量
   int length;                       //实际元素个数
public:
   Array(int n=1)                    //构造函数
   {  data=new T[n];                 //为data分配指向的内存空间，n为容量
      length=0;                      //实际元素个数为0
   }
   ~Array()                          //析构函数
   {
      delete [] data;                //释放data指向的空间
   }
   void add(T x)                     //添加一个元素x
   {  data[length]=x;
      length++;
   }
   void display()                    //输出data指向的所有元素
   {  for (int i=0;i<length;i++)
         cout << data[i] << " ";
      cout << endl;
   }
};
```

```
int main()
{  Array<char> ac(3);                    //定义模板类的对象ac
   ac.add('x'); ac.add('y'); ac.add('z');
   cout << "ac: ";
   ac.display();                         //输出: x y z

   Array<int> ai(5);                     //定义模板类的对象ai
   ai.add(1); ai.add(2); ai.add(3);
   ai.add(4); ai.add(5);
   cout << "ai: ";
   ai.display();                         //输出: 1 2 3 4 5
   return 0;
}
```

说明

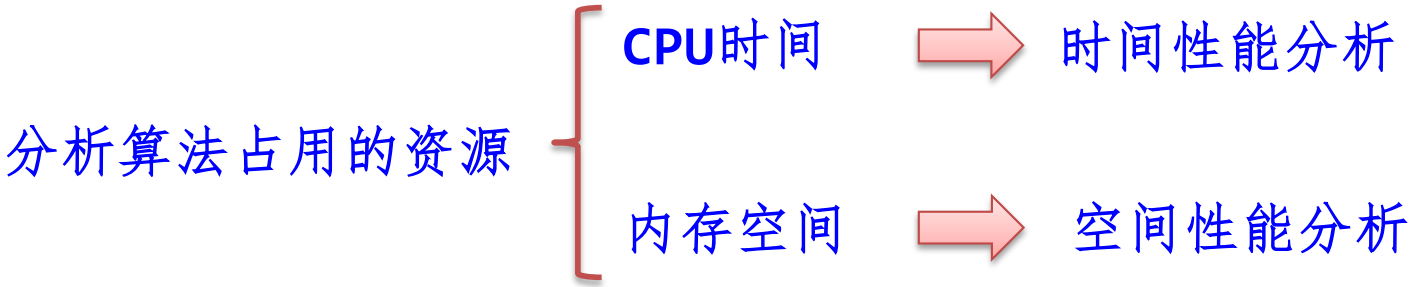
数据结构的实现通常用类模板来描述：

- 数据结构关注的是数据元素及其关系是如何保存的，基于这些关系的运算是如何实现的，而数据元素可以是任意类型。
- 使用类模板来描述，可以避免对于具体数据元素类型的依赖。

1.3 算法分析

1.3.1 算法的设计目标

- 正确性。
- 可使用性。
- 可读性。
- 健壮性。
- **高时间性能与低存储量需求。**



算法分析目的：分析算法的时空效率以便改进算法性能。

1.3.2 算法时间性能分析

算法分析方式：

❶ **事后分析统计方法**：编写算法对应程序，统计其执行时间。

- 编写程序的语言不同
- 执行程序的环境不同
- 其他因素

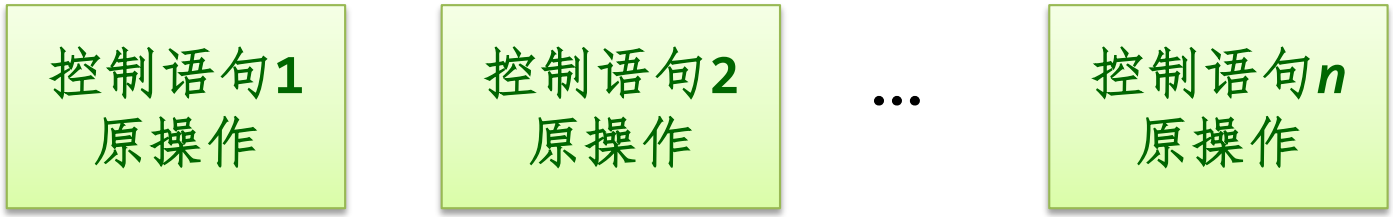
所以不能用绝对执行时间进行比较。

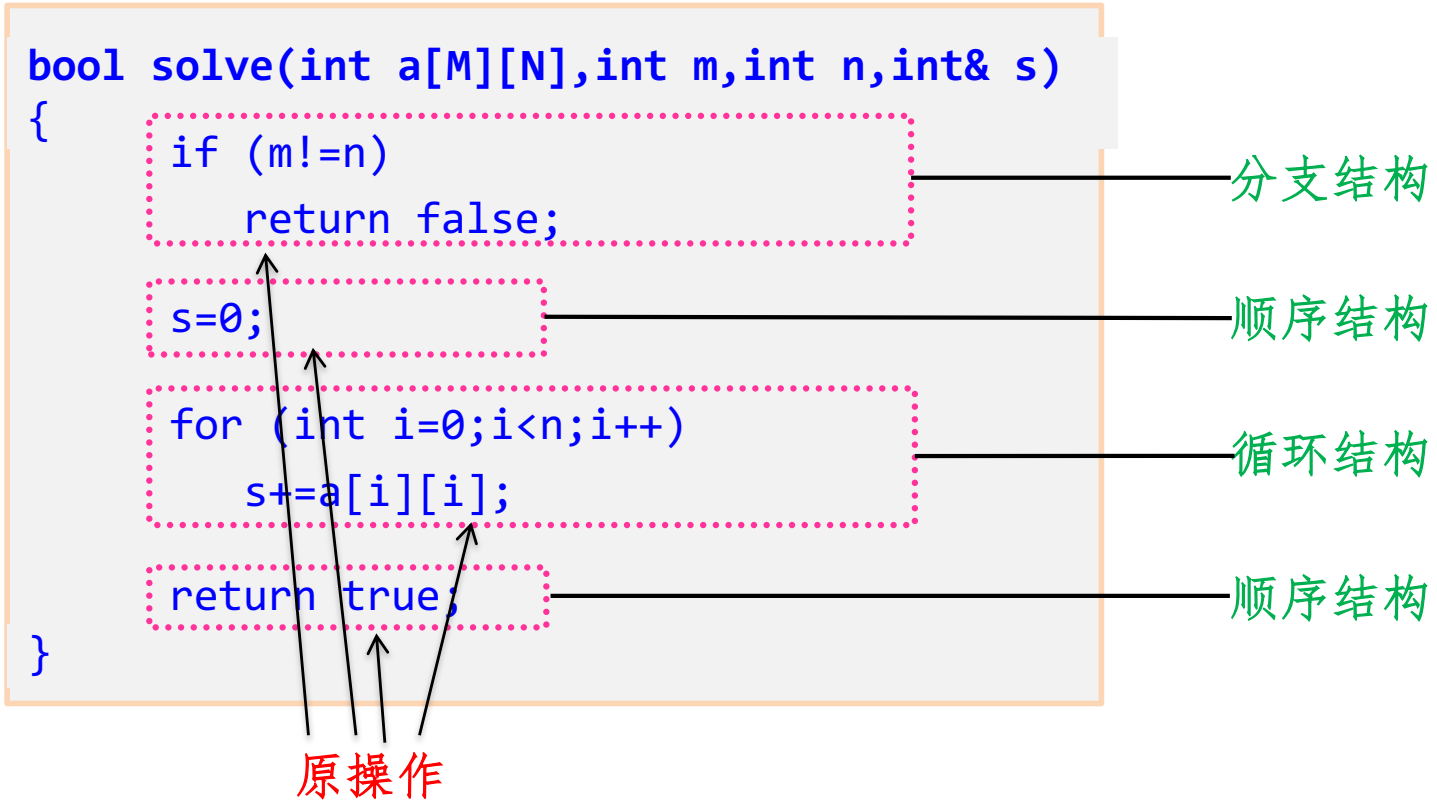
❷ **事前估算分析方法**：撇开上述因素，认为算法的执行时间是问题规模*n*的函数。 ✓

1．分析算法的时间复杂度

一个算法是由控制结构（顺序、分支和循环三种）和原操作（指固有数据类型的操作，如+、-、*、/、++和--等）构成的。算法执行时间取决于两者的综合效果。

一个算法的基本构成：





- 算法的执行时间取决于控制结构和原操作的综合效果。
- 在一个算法中，执行原操作的次数越少，其执行时间也就相对地越少；执行原操作次数越多，其执行时间也就相对地越多。
- 算法中所有原操作的执行次数称为**算法频度**，这样一个算法的执行时间可以由算法频度来计量。

1）计算算法频度T(*n*)

- 假设算法的问题规模为*n*，例如对10个整数排序，问题规模为*n*就是10。算法频度是问题规模*n*的函数，用T(*n*)表示。
- 算法执行时间大致等于原操作所需的时间×T(*n*)，也就是说T(*n*)与算法的执行时间成正比。为此用T(*n*)表示算法的执行时间。比较不同算法的T(*n*)大小得出算法执行时间的好坏。

【例1.10】求两个n阶方阵相加$C=A+B$的算法如下，求$T(n)$。

```
void matrixadd(int A[N][N],int B[N][N],int
                        C[N][N],int n)
{  for (int i=0;i<n;i++)                  //语句①
     for (int j=0;j<n;j++)                //语句②
         C[i][j]=A[i][j]+B[i][j];         //语句③
}
```

执行次数

语句①：$n+1$

语句②：$n(n+1)$

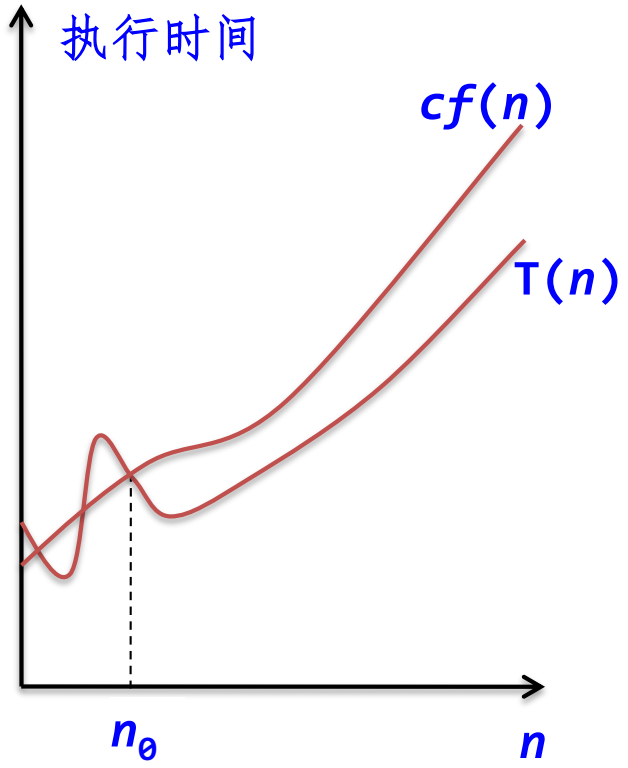
语句③：n^2

$$T(n)=n+1+n(n+1)+n^2$$
$$=2n^2+2n+1。$$

2）T(n)采用时间复杂度表示

算法中执行时间T(n)是问题规模n的某个函数f(n)，记作：

$$T(n) = O(f(n))$$



“O”的形式定义为：

$T(n) = O(f(n))$表示存在一个正的常数c，使得当$n \geq n_0$时都满足：

$$|T(n)| \leq c|f(n)|$$

↑

$f(n)$是$T(n)$的上界

↑

这种上界可能很多，通常取最接近的上界，即紧凑上界

大致情况：

$$\lim_{n\to\infty} \frac{T(n)}{f(n)} = c$$

本质上讲，是一种T(*n*)
最高数量级的比较

也就是只求出T(*n*)的最高阶，忽略其低阶项和常系数，这样既可简化T(*n*)的计算，又能比较客观地反映出当*n*很大时算法的时间性能。

例1.10

```
void matrixadd(int A[N][N],int B[N][N],int C[N][N],int n)
{  for (int i=0;i<n;i++)                          //语句①
     for (int j=0;j<n;j++)                        //语句②
         C[i][j]=A[i][j]+B[i][j];                 //语句③
}
```

$T(n)=2n^2+2n+1=O(n^2)$

时间复杂度

一般地：

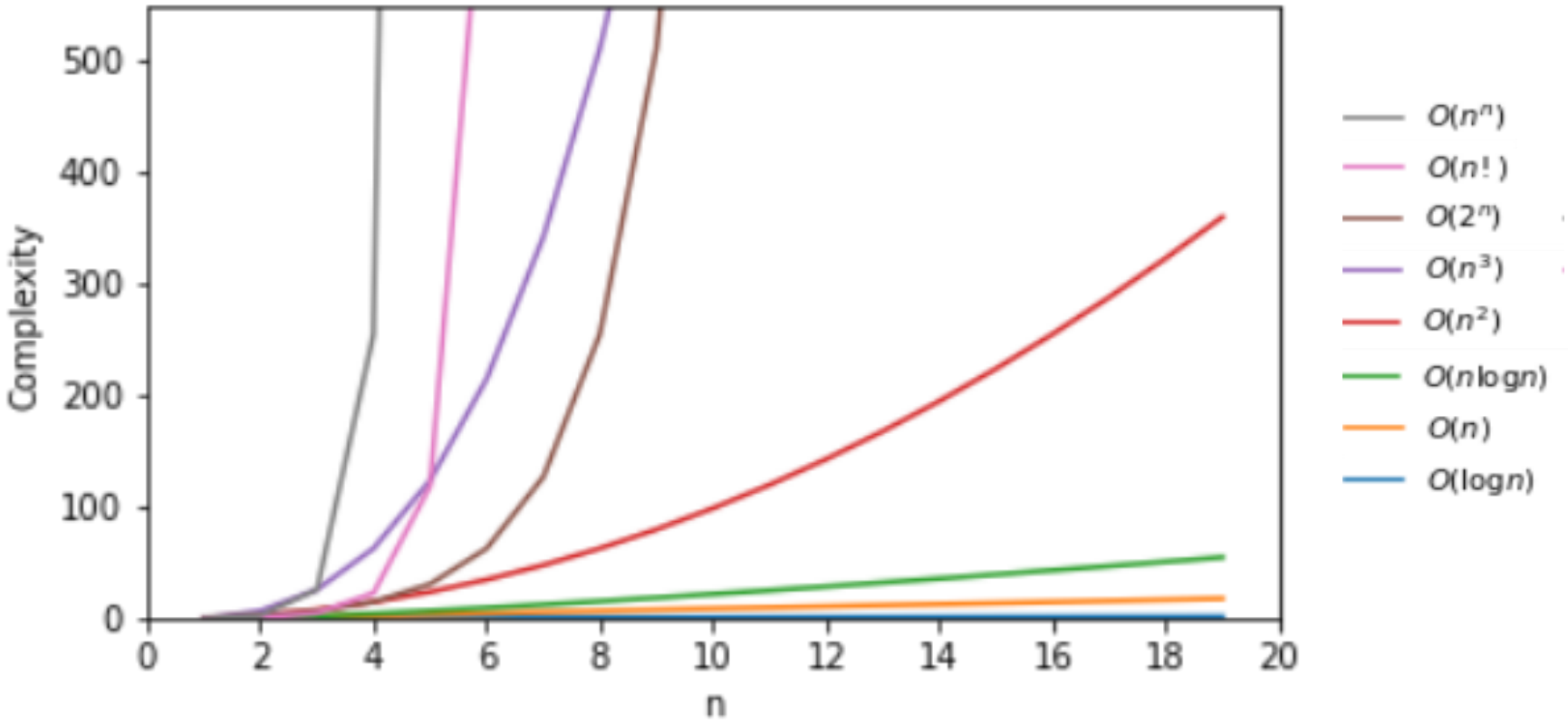
- 一个没有循环的算法的执行时间与问题规模n无关，记作$O(1)$，也称作常数阶。
- 一个只有一重循环的算法的执行时间与问题规模n的增长呈线性增大关系，记作$O(n)$，也称线性阶。
- 其余常用的算法时间复杂度还有平方阶$O(n^2)$、立方阶$O(n^3)$、对数阶$O(\log_2 n)$、指数阶$O(2^n)$等。

各种不同算法时间复杂度的比较关系如下：

$O(1)<O(\log_2 n)<O(n)<O(n\log_2 n)<O(n^2)<O(n^3)<O(2^n)<O(n!)<O(n^n)$

多项式阶：
P问题

指数阶：
NP问题



```
int Sum1(int n)
{  int i,s=0;
   for (i=1;i<=n;i++)
      s+=i;
   return s;
}
```

```
int Sum2(int n)
{  int s;
   s=n*(n+1)/2;
   return s;
}
```

O(*n*)

O(1)

Sum2更好

3）简化的算法时间复杂度分析

- 一种简化的算法时间复杂度分析方法是，仅仅考虑算法中的基本操作。
- 所谓基本操作是指算法中最深层循环内的原操作。而算法执行时间大致等于基本操作所需的时间×其运算次数。所以在算法分析时，计算T(*n*)时仅仅考虑基本操作的执行次数。

例1.10

```
void matrixadd(int A[N][N],int B[N][N],int C[N][N],int n)
{  for (int i=0;i<n;i++)                    //语句①
     for (int j=0;j<n;j++)                  //语句②
         C[i][j]=A[i][j]+B[i][j];           //语句③
}
```

基本操作，执行次数为n^2

$T(n)=n^2=O(n^2)$

【例1.11】分析以下算法的时间复杂度。

```
int fun(int n)
{  int s=0
   for (int i=0;i<=n;i++)
      for (int j=0;j<=i;j++)
         for (int k=0;k<j;k++)
            s++;
   return s;
}
```

基本操作

算法频度为：

$$T(n)=\sum_{i=0}^{n}\sum_{j=0}^{i}\sum_{k=0}^{j-1}1=\sum_{i=0}^{n}\sum_{j=0}^{i}(j-1-0+1)=\sum_{i=0}^{n}\sum_{j=0}^{i}j$$

$$=\sum_{i=0}^{n}\frac{i(i+1)}{2}=\frac{1}{2}\left(\sum_{i=0}^{n}i^2+\sum_{i=0}^{n}i\right)=\frac{2n^3+6n^2+4n}{12}=O(n^3)$$

【例1.12】分析以下算法的时间复杂度。

```
int fun(int n)
{  int x=2;
   while (x<n/2)
     x=2*x;
   return x;
}
```

- 基本操作是语句x=2*x，设该语句执行m次。
- 则$2^{m+1} \geqslant n/2$（该条件刚成立时while循环结束），不妨添加一个常量k使得$2^{m+1}=n/2+k$成立，故$m=\log_2(n/2+k)-1$。
- $T(n)=m=\log_2(n/2+k)-1=O(\log_2 n)$。
- 为了简单，可以直接认为$2^{m+1}=n/2$成立。求出 $T(n)=m=\log_2(n/2)-1=\log_2 n-2=O(\log_2 n)$。

4）时间复杂度的求和、求积定理

求和定理：假设$T_1(n)$和$T_2(n)$是程序段P_1、P_2的执行时间，并且有$T_1(n)=O(f(n))$，$T_2(n)=O(g(n))$。

那么先执行P_1，再执行P_2的总执行时间是

$T_1(n)+T_2(n)=O(MAX(f(n), g(n)))$，即总的时间复杂度=量级最大的程序段的时间复杂度，如多个并列循环就属于这种情况。

```
int fun(int n)
{  int s=0;
   for (int i=0; i<n; i++)    //第一个for循环
       s+=2;
   for (int i=0; i<n; i++)    //第二个for循环
       for (int j=0; j<n; j++)
           s++;
   return s;
}
```

$T(n)=O(MAX(n, n^2))=O(n^2)$。

求积定理：假设$T_1(n)$和$T_2(n)$是程序段P_1、P_2的执行时间，并且有$T_1(n)=O(f(n))$，$T_2(n)=O(g(n))$。

那么，$T_1(n)\times T_2(n)=O(f(n)\times g(n))$，如嵌套代码的复杂度=嵌套内外代码复杂度的乘积。

```
int fun(int n)
{  int s=0;
   for (int i=0; i<n; i++)    //第一个for循环
       s+=2;
   for (int i=0; i<n; i++)    //第二个for循环
       for (int j=0; j<n; j++)
           s++;
   return s;
}
```

$T2(n)=O(n)\times O(n)=O(n^2)$

2．算法的最好、最坏和平均时间复杂度

定义：设一个算法的输入规模为n，D_n是所有输入实例的集合，任一输入$I \in D_n$，$P(I)$是I出现的概率，有 $\sum_{I \in D_n} P(I) = 1$ ，$T(I)$是算法在输入I下的执行时间，则算法的平均时间复杂度为：

$$A(n) = \sum_{I \in D_n} P(I) \times T(I)$$

例如，10个1～10的整数序列递增排序：

$n=10$

$I_1=\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

$I_2=\{2, 1, 3, 4, 5, 6, 7, 8, 9, 10\}$

…

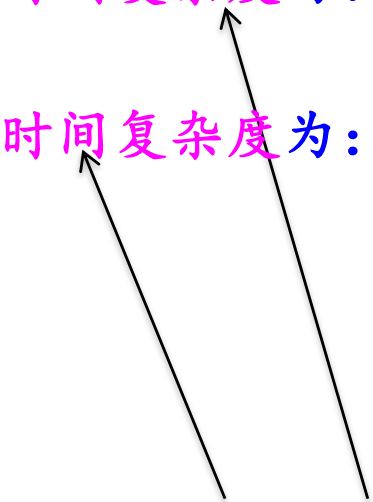
$I_m=\{10, 9, 8, 7, 6, 5, 4, 3, 2, 1\}$

} 构成D_n，$P(I)=1/m$

所有可能的初始序列有m个，$m=10!$

算法的最坏时间复杂度为：$W(n)=\max_{I\in D_n}\{T(I)\}$

算法的最好时间复杂度为：$B(n)=\min_{I\in D_n}\{T(I)\}$



一种或几种特殊情况

算法时间性能比较：假如求同一问题有两个算法：*A*和*B*，如果算法*A*的平均时间复杂度为$O(n)$，而算法*B*的平均时间复杂度为$O(n^2)$。

一般情况下，认为算法*A*的时间性能好比算法*B*。

【例1.13】以下算法用于在数组a[0..n-1]查找元素k，假设k总是包含在a中，分析算法的最好、最坏和平均时间复杂度。

```
int findk(int a[],int n,int k)
{  int i=0;
   while (i<n && a[i]!=k)
      i++;
   return i;
}
```

```
int findk(int a[],int n,int k)
{  int i=0;
   while (i<n && a[i]!=k)
      i++;
   return i;
}
```

该算法的主要时间花费在元素比较上，可以将元素比较看成基本操作。

（1）算法在查找中总是从$i=0$开始的，如果$a[0]=k$，则仅仅一次比较就成功找到k，呈现最好情况，所以算法的最好时间复杂度为$O(1)$。

```
int findk(int a[],int n,int k)
{  int i=0;
   while (i<n && a[i]!=k)
      i++;
   return i;
}
```

（2）如果*a*[*n*-1]=*k*，则需要*n*次比较成功找到*k*，呈现最坏情况，所以算法的最坏时间复杂度为O(*n*)。

```
int findk(int a[],int n,int k)
{  int i=0;
   while (i<n && a[i]!=k)
      i++;
   return i;
}
```

（3）考虑平均情况：

a[0]=k时比较1次

a[1]=k时比较2次

…

a[n-1]=k时比较n次

共n种情况，假设等概率，也就是说每种情况的概率为1/n，则平均比较次数=(1+2+…+n)/n=(n+1)/2=O(n)，所以算法平均时间复杂度为O(n)。

1.3.3 算法空间性能分析

- 一个算法的存储量包括形参所占空间和临时变量所占空间。在对算法进行存储空间分析时，只考察临时变量所占空间。
- 空间复杂度是对一个算法在运行过程中临时占用的存储空间大小的量度，一般也作为问题规模n的函数，以数量级形式给出，记作：$S(n)=O(g(n))$。
- 其中“O”的含义与时间复杂度分析中的相同。

1.3.3 算法空间性能分析

为什么空间复杂度分析只考虑临时变量？

- 完整的空间消耗包含：
  - 算法代码段占用的固定空间（编译器优化）
  - 输入数据占用的必要空间（问题固有）
  - 算法运行过程中产生的临时存储空间（算法的空间优化能力）
    - ✓ 临时变量（如循环计数器、中间计算结果）
    - ✓ 动态分配的数据结构（如递归调用栈、辅助数组）
    - ✓ 函数调用的栈空间（递归深度越大，栈空间占用越多）

各种内排序方法的比较和选择

排序方法	时间复杂度			空间复杂度
	平均情况	最坏情况	最好情况	
直接插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$
折半插入排序	$O(n^2)$	$O(n^2)$	$O(n\log_2 n)$	$O(1)$
希尔排序	$O(n^{1.58})$			$O(1)$
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(\log_2 n)$
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$
二路归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$

```
int Max(int a[],int n)
{   int maxi=0;
    for (int i=1;i<n;i++)
        if (a[i]>a[maxi])
            maxi=i;
    return a[maxi];
}
```

函数体内分配的变量空间为临时空间，不计形参占用的空间，这里的仅计**i**、**maxi**变量的空间。

为什么算法空间分析只考虑临时空间，而不必考虑形参的空间呢？

```
int Max(int a[],int n)
{  int maxi=0;
   for (int i=1;i<n;i++)
      if (a[i]>a[maxi])
         maxi=i;
   return a[maxi];
}
```

如果Max函数中再考虑形参*a*的空间，就重复累计了执行整个算法所需的空间。

传递数组地址

```
void Maxfun()
{  int b[]={1,2,3,4,5};
   int n=5;
   printf("Max=%d\n",Max(b,n));
}
```

Maxfun算法中为*b*数组分配了相应的内存空间，其空间复杂度为O(*n*)

【例1.14】分析例1.10～例1.13算法的空间复杂度。

例1.10

```
void matrixadd(int A[M][N],int B[M][N],int C[M][N],int n)
{  for (int i=0;i<n;i++)                          //语句①
     for (int j=0;j<m;j++)                        //语句②
         C[i][j]=A[i][j]+B[i][j];                 //语句③
}
```

空间复杂度为O(1)

例1.11

```
int fun(int n)
{  int s=0
   for (int i=0;i<=n;i++)
      for (int j=0;j<=i;j++)
         for (int k=0;k<j;k++)
            s++;
   return s;
}
```

空间复杂度为O(1)

例1.12

```
int fun(int n)
{  int x=2;
   while (x<n/2)
     x=2*x;
   return x;
}
```

空间复杂度为O(1)

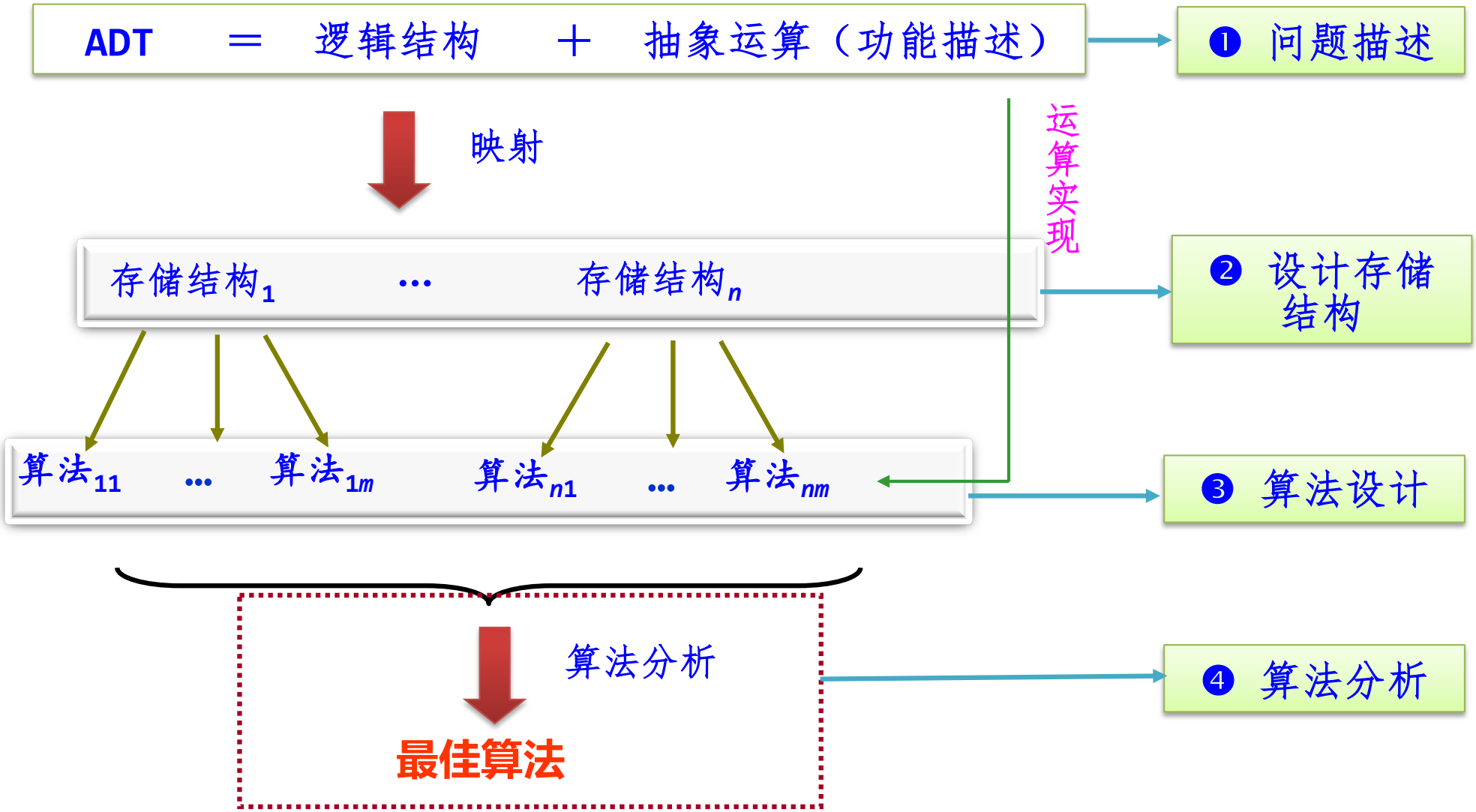
例1.13

```
int findk(int a[],int n,int k)
{  int i=0;
   while (i<n && a[i]!=k)
      i++;
   return i;
}
```

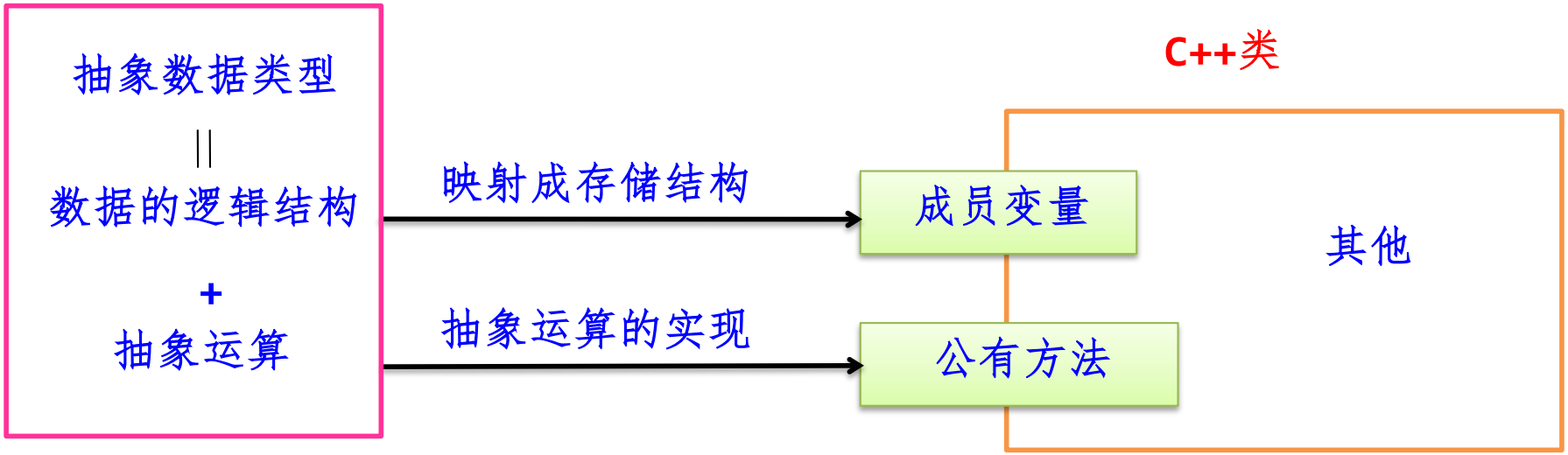
空间复杂度为O(1)

1.4 数据结构的目标

好算法设计的过程



采用C++语言实现抽象数据类型时，通常将一个抽象数据类型设计成一个C++类，采用类的数据变量表示数据的存储结构，将抽象运算通过类的公有方法实现。



存储结构对算法的影响主要在两方面：

- 存储结构的存储能力
- 存储结构应与所选择的算法相适应

【例1.15】设计一个完整的程序实现例1.6的抽象数据类型，并用相关数据进行测试。

📖 问题描述

【例1.6】构造集合ADT Set，假设其中元素为整型，遵循标准数学定义，基本运算包括：

求集合长度、求第i个元素、判断一个元素是否属于集合、向集合中添加一个元素、从集合中删除一个元素、复制集合和输出集合中所有元素。

另外增加3个集合运算：

求两个集合并Union、集合交Inter和集合差Diff。

```
ADT Set                                          //集合的抽象数据类型
{   数据对象:
      data={di | 0≤i≤size-1 }                    //存放集合中元素
    数据关系:
      无
    基本运算:
       getsize()                                 //返回集合的长度
       get(int i)                                //返回集合的第i个元素
       IsIn(E e)                                 //判断e是否在集合中
       add(E e)                                  //将元素e添加到集合中
       delete(E e)                               //从集合中删除元素e
       Copy(s)                                   //返回当前集合的复制集合
       display()                                 //输出集合中的元素
       Union(Set s2)                             //求s3=s1Us2 (s1为当前集合)
       Inter(Set s2)                             //求s3=s1∩s2 (s1为当前集合)
       Diff(Set s2)                              //求s3=s1-s2 (s1为当前集合)
} ADT Set
```

设计存储结构

```
class Set                    //集合类
{   int* data;               //data存放集合元素
    int length;              //length为集合的长度
    …
};
```

设计运算算法

Set类包含以下基本运算方法：

```
public:
  Set()                                  //构造函数
  {  data=new int[MaxSize];              //data存放集合元素
     length=0;                           //length为集合的长度
  }

  ~Set()                                 //析构函数
  {
     delete[] data;
  }
```

```
int getlength()                          //返回集合的长度
{
   return length;
}

int get(int i)                           //返回集合的第i个元素
{  if (i<0 || i>=length)                 //检测参数i的正确性
     throw "参数i错误";
   return data[i];
}

bool IsIn(int e)                         //判断e是否在集合中
{  for (int i=0;i<length;i++)
      if (data[i]==e) return true;
   return false;
}
```

```
void add(int e)                          //将元素e添加到集合中
{  if (!IsIn(e))                         //元素e不在集合中
   {  data[length]=e;
      length++;
   }
}

void delelem(int e)                      //从集合中删除元素e
{  int i=0;
   while (i<length && data[i]!=e) i++;
   if (i>=length)
       return;                           //未找到元素e直接返回
   for (int j=i+1;j<length;j++)          //找到元素e后通过移动实现删除
       data[j-1]=data[j];
   length--;
}
```

```
Set& Copy()                          //返回当前集合的复制集合
{  static Set s1;
   for (int i=0;i<length;i++)
      s1.data[i]=data[i];
   s1.length=length;
   return s1;
}

void display()                       //输出集合中的元素
{  for (int i=0;i<length-1;i++)
      printf("%d ",data[i]);
   printf("%d\n",data[length-1]);
}
```

s3=s1Us2:

- s3=s1
- 将s2中不属于s1的元素添加到s3

```
Set& Union(Set& s2)              //求s3=s1Us2(s1为当前集合)
{  Set& s3=Copy();               //将当前集合复制到s3
   for (int i=0;i<s2.getlength();i++)
                                 //将s2中不在s1中的元素添加到s3中
   {  int e=s2.get(i);
      if (!IsIn(e)) s3.add(e);
   }
   return s3;                    //返回s3
}
```

s3=s1∩s2:

- s3={}
- 将s1中属于s2的元素添加到s3

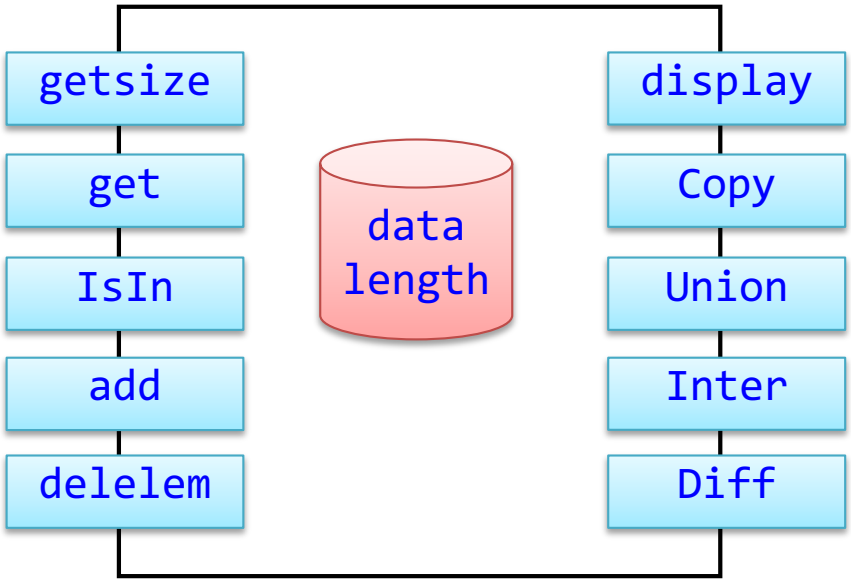
```
Set& Inter(Set& s2)              //求s3=s1∩s2(s1为当前集合)
{  static Set s3;
   for (int i=0;i<length;i++)
                                 //将s1中出现在s2中的元素复制到s3中
   {  int e=data[i];
      if (s2.IsIn(e)) s3.add(e);
   }
   return s3;                    //返回s3
}
```

s3=s1-s2:

- s3={}
- 将s1中不属于s2的元素添加到s3

```
Set& Diff(Set& s2)                  //求s3=s1-s2(s1为当前集合)
{  static Set s3;
   for (int i=0;i<length;i++)//将s1中不出现在s2中的元素复制到s3中
   {  int e=data[i];
      if (!s2.IsIn(e)) s3.add(e);
   }
   return s3;                       //返回s3
}
```

Set类的描述


getsize
get
IsIn
add
delelem
data
length
display
Copy
Union
Inter
Diff

设计主程序

```
int main()
{  Set s1,s2;
   s1.add(1);
   s1.add(4);
   s1.add(2);
   s1.add(6);
   s1.add(8);
   printf("集合s1: "); s1.display();
   printf("s1的长度为%d\n",s1.getlength());
   s2.add(2);
   s2.add(5);
   s2.add(3);
   s2.add(6);
   printf("集合s2: "); s2.display();
```

```
    printf("集合s1和s2的并集->s3\n");
    Set& s3=s1.Union(s2);
    printf("集合s3: "); s3.display();
    printf("集合s1和s2的差集->s4\n");
    Set& s4=s1.Diff(s2);
    printf("集合s4: "); s4.display();
    printf("集合s1和s2的交集->s5\n");
    Set& s5=s1.Inter(s2);
    printf("集合s5: "); s5.display();
    return 0;
}
```

程序执行结果

```
集合s1: 1 4 2 6 8
s1的长度为5
集合s2: 2 5 3 6
集合s1和s2的并集->s3
集合s3: 1 4 2 6 8 5 3
集合s1和s2的差集->s4
集合s4: 1 4 8
集合s1和s2的交集->s5
集合s5: 2 6
```